Issues — Open Workable Items

Revision: 52 **Last modified:** 2026-08-22T08:38:16Z **Ticket prefix:** `BOB` (operator-mandated, 2026-06-06) **Scope:** Open/active items only. Closed items migrate to `Fixed.md`.

> Tracking: this file + `Issues_Summary.md` are authoritative for open work. The SQLite single-source-of-truth + `docs_chain` engine (BOB-010) is complete.

BOB-008 — RuTracker automated login blocked by CAPTCHA

Status: Operator-blocked **Type:** Bug **Created:** 2026-06-06 **Operator-Block-Details:** WHAT — RuTracker login with stored creds returns no session cookie (CAPTCHA wall). WHY — automated user/pass login is CAPTCHA-gated; self-resolution exhausted (creds correct + wired, login attempted, `auth=True`). UNBLOCK — [A] operator completes the CAPTCHA flow at `/api/v1/auth/rutracker/captcha` + `/login`. [B] operator pastes a fresh `bb_session` cookie via `/auth/rutracker/cookie-login`. WHO — operator.

Evidence: live search per-tracker stat `rutracker status=error auth=True`. The diagnostic now reports `error_type="upstream_captcha"` with the FACT-based message “rutracker login.php returned no session cookie — this is the rutracker anti-abuse CAPTCHA wall (gates login.php when logins spike), not a credential failure. Set RUTRACKER_COOKIES from a logged-in browser session to bypass the login round-trip.” (`download-proxy/src/merge_service/search.py`). The earlier quote here — `error="login returned no session cookie — likely CAPTCHA"` — was SUPERSEDED when BOB-117 landed and no longer exists in source; it is corrected rather than left as stale evidence (§11.4.6 no-guessing, §11.4.7 evidence must reflect the conditions it claims).

BOB-065 — Lava P2: Egress diagnosis and VPN-host SOCKS routing (containers pkg/egress)

Status: Queued **Type:** Task **Severity:** High

[Backfill from RD2-15/GA-05, audit doc 2026-08-08] Lava-porting finding P2 (Egress decision + VPN-host routing, Lava PLAYBOOK sections 0 and 4). Problem Boba-Base shares: on a datacenter host, trackers are network-blocked (DNS-fail/TLS-MITM, not Cloudflare so FlareSolverr cannot fix). Affects Jackett indexer fetches + merge_service/download-proxy + plugin engines. Diagnosis (port the script): curl https://api.ipify.org (host IP) + curl -o /dev/null -w %{http_code} https:/// direct vs via a VPN-host SOCKS proxy. Different egress IP + 200 via proxy confirms. Fix: route outbound through a VPN-connected host (the nezha pattern). SOCKS tunnel ssh -D 127.0.0.1:1080 -N (use –socks5-hostname for remote DNS); point Jackett + download-proxy + qBitTorrent-go at it (P3). For browser-cookie harvest, run the harvester ON the VPN host. Port: containers submodule pkg/egress (tunnel up/verify) + scripts/egress-via-vpn.sh glue; reuse Boba-Base existing ensure-macos-tunnel.sh style. TDD: assert the via-proxy egress IP != direct host IP AND a known-blocked tracker returns 200 via proxy. Source: docs/PORTING-FROM-LAVA.md. Per audit RD2-15 [P0]: Create tracked workable items (BOB-064..067) for the four Lava-porting findings, citing implementing commits as evidence, closed as Implemented.

BOB-066 — Lava P3: BOBA_UPSTREAM_PROXY in download-proxy + qBitTorrent-go + Jackett + compose env-forward

Status: In progress **Type:** Task **Severity:** High

[Backfill from RD2-15/GA-05, audit doc 2026-08-08] Lava-porting finding P3 (Configurable outbound proxy in the services, Lava PLAYBOOK section 3). download-proxy (Python): httpx/requests honor HTTP_PROXY/HTTPS_PROXY/ALL_PROXY/NO_PROXY env natively — add an explicit BOBA_UPSTREAM_PROXY config that sets these for tracker-bound clients, with loopback bypass (NO_PROXY=127.0.0.1,localhost,jackett). qBitTorrent-go: set http.Transport.Proxy (socks5 native, remote DNS) from a BOBA_UPSTREAM_PROXY env — mirror Lava internal/httpx/proxy.go. Jackett: has a built-in proxy setting (configure via its API/ServerConfig). Deploy gotcha (port): the env must be FORWARDED into the containers (docker-compose.yml env / the

boba-ctl deploy) — Lava bug was a missing allow-list entry. Verify on distroless via podman inspect, not exec printenv. TDD: a test with a local proxy asserts the service tracker request traverses it; falsifiability: disable the wiring so the test fails. Source: docs/PORTING-FROM-LAVA.md. Per audit RD2-15 [P0]: Create tracked workable items (BOB-064..067) for the four Lava-porting findings, citing implementing commits as evidence, closed as Implemented.

BOB-068 — RD2-00: unattributed, unreviewed Auto-commit mechanism pushing to main

Status: In progress **Type:** Bug **Severity:** High

RD2-00: unattributed, unreviewed Auto-commit mechanism pushing to main

[BOB-136 adoption audit 2026-08-21 -> In progress] Work landed but acceptance NOT met. a7e55f9 completed the Phase-1 investigation (no daemon; shared-checkout race confirmed) and 0972cbc added commit-push-all.sh –scope + challenge, described in its own message as an ‘Interim tooling remedy for BOB-068 while the §11.4.179 architectural fix (task #67 proposal) is designed’. 35e53db is a DRAFT PROPOSAL only. The §11.4.179 clone-isolation fix is not landed, so the item stays open.

BOB-069 — RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape backfill

Status: In progress **Type:** Task **Severity:** High

RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape backfill

[BOB-136 adoption audit 2026-08-21 -> In progress] a4173c8 stood up the ledger + 4 followups; 709b06c closed 2 Important review findings (CODEGRAPH-1.5.0-GITIGNORE entry, BOB-108 filed). The BOB-009/010 evidence_path gap this item names as remaining open was closed under BOB-086 (f78f383). NOT terminal by the item’s own text: it is explicitly scoped ‘P1-ongoing’, with the full retroactive audit of the ~50 GA-NN/RD2-NN findings deliberately NOT attempted.

BOB-074 — RD2-07: DDoS-class testing fully absent from the mandated test-type matrix

Status: In progress **Type:** Task **Severity:** Medium

RD2-07: DDoS-class testing fully absent from the mandated test-type matrix

[BOB-136 adoption audit 2026-08-21 -> In progress] ae2b5cb authored challenges/scripts/ddos_resilience_challenge.sh (424 lines, 3 public surfaces) + docs/testing/{ddos_resilience,test_type_matrix}.md, so DDoS-class testing is no longer 'fully absent'. But of the three coverages this item's Fix line enumerates, a grep of the shipped challenge finds flood=2 hits, malformed=0, exhaust=0 — malformed-request-flood and resource-exhaustion-under-attack are not covered. 258b7db filed the 6 followups (BOB-109..114). Partial, not complete.

BOB-077 — RD2-10: Identify second host running the Auto-commit rsync/sync mechanism (OPERATOR-DECISION)

Status: Operator-blocked **Type:** Task **Operator-Block-Details:** WHAT: OPERATOR must identify which second host holds push credentials + confirms whether the rsync/sync mechanism is intentional or a stale job to retire. This session cannot inspect a host it has no access to (§11.4.6/§11.4.101). WHY: This session cannot inspect or reach the +0500 host that produced the Auto-commit fast-forwards (§11.4.6 no-guessing, §11.4.101 reversible-safe default). UNBLOCK: [A] operator names the second host + confirms intentional (proceed to RD2-11 wiring) · [B] operator confirms stale/misconfigured (retire the job) · [C] operator confirms the second live Claude session/device is the source (Rev-6 Update 2 root cause, proceed to per-session discipline enforcement) WHO: Operator **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-10, P0 OPERATOR-DECISION] Root-caused as far as this host allows (see RD2-00 Update): reflog proves the two newest commits arrived via plain pull: Fast-forward from a +0500 host, matching this project established 2026-06-28 cross-host rsync-sync pattern (cdb555f/55b8671). Needs operator input to go further — which second host runs it, and is it the intended mechanism (just needs a real message + review gate) or a stale job to retire. Not auto-executed (§11.4.6/§11.4.101 — this session cannot inspect or guess at a host it has no access to). Rev-6

update: root cause narrowed to a second live Claude session (Opus 5, same +0500 host) that independently landed constitution 177f2b0. Priority: P0 (operator-blocked).

BOB-078 — RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire it

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-11, P0] Once identified (RD2-10): either wire it through a real §11.4.234 dedicated commit/push script (with the hook-validation stages this project already has via .claude/settings.json PreToolUse guard, extended to a real pre-commit content check) or shut it down if it serves no purpose. Priority: P0. Blocked on RD2-10.

BOB-080 — RD2-13: Retroactive Fable-xhigh code review of the substantive Auto-commit diffs

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-13, P1] Retroactively run the mandatory independent Fable-xhigh code review (§11.4.125/§11.4.142/§11.4.209) against the substantive diffs the Auto-commit mechanism introduced (start.sh three new functions especially, since they have zero test coverage — see Root Cause 4 / RD2-24) — even though the changes already landed, a post-hoc review closes the governance gap and will surface RD2-24 (GA-27 missing tests) if not already caught. Priority: P1.

BOB-082 — RD2-15: Create BOB-064..067 workable items for the four Lava-porting findings (closes GA-05)

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-15, P0 — closes GA-05] Create tracked workable items (BOB-064..067, via the workable-items tool — never raw MD edits) for the four Lava-porting findings, citing implementing commits as evidence, closed as Implemented. GA-05 evidence: grep -in lava|BOB-06[4-7] docs/Issues.md docs/Fixed.md returns zero hits. Priority: P0. NOTE: BOB-064..067 have been created 2026-08-10 as Task/Queued (the four Lava P1..P4 items); the closed-as-Implemented status flip (citing per-finding implementing commits) remains owed under this item.

BOB-085 — RD2-18: Create top-level Boba proxy/merge-service v1.0.0 readiness ledger (closes GA-10)

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-18, P2 — closes GA-10] Create the top-level Boba (proxy/merge-service) v1.0.0 readiness ledger (GA-10) — dedupe with the browser_extension existing one as the template. GA-10 evidence: only docs/RELEASE_READINESS_20260616.html/.md/.pdf (dated point-in-time snapshot) and the extension own ledger exist; no top-level proxy/merge-service ledger created. Priority: P2.

BOB-087 — RD2-20: Wire docs_chain / commit-seam sync hook per §11.4.106(F) so DB writes cannot land without MD mirror

Status: Completed (→ Fixed.md) **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-20, P0] Wire (or fix) the docs_chain / commit-seam sync hook per §11.4.106(F) so a docs/workable_items.db write can never again land without its MD mirror in the same commit — this is the mechanical fix that prevents Root Cause 2 from recurring, not just a one-time catch-up. Priority: P0.

Progress 2026-08-21: MEASURED against all three seams §11.4.106(F) names, not one. COMMIT seam: COVERED — scripts/hooks/docs-sync-commit-seam.sh is invoked from scripts/commit-push-all.sh at BOTH commit call sites (the –scope branch and the git add -A branch) via _docs_sync_seam_check, after staging and before git commit, exiting 1 on refusal; there is no third path to git commit in that script. Proven in BOTH directions on temp copies with the real DB sha256 unchanged: an MD-side

body edit is detected and named (self-test golden-bad, victim BOB-008), and a real engine DB write with the Markdown deliberately left stale is detected and named (golden-bad, BOB-084) — that second direction is the one the item's own text claims — while a clean tree stays silent (negative control, no §11.4.201(1) false positive). BUILD seam: COVERED — pre_build_verification.sh invariant 17 runs validate AND diff with –issues/–fixed passed explicitly (never the flagless form BOB-155 fixed), invariants 18/22 cover the export leg with a real-invocation assertion, invariant 24 runs the real docs_chain engine verify –all; RESIDUAL: no CHECK 3 equivalent there, so the build seam inherits diff's blindness to the body_md class (the BOB-136 class). CONSTITUTION-PULL seam: NOT COVERED — a grep for workable-items|docs-sync|docs_chain|11.4.106 returns 0 in BOTH constitution/scripts/post_update_hook.sh and scripts/verify-all-constitution-rules.sh, control-needled so the zeros are sight not blindness (needle 'skill' 38 hits, 'covenant_propagation_suite' 7 hits, negative control 0); of the 172 gates under constitution/scripts/gates/ only two mention the engines and both are anchor-literal presence gates that compare no DB against any Markdown, and config/constitution-sweep.conf adds no such check. So a constitution pull can be treated as canonical with the tracker never re-compared. REMAINS: wire the already-existing seam into scripts/verify-all-constitution-rules.sh BY REFERENCE (bash scripts/hooks/docs-sync-commit-seam.sh –files docs/Issues.md docs/Fixed.md docs/workable_items.db), reporting PASS/FAIL/SKIP-with-reason in the sweep's own vocabulary and never a silent pass on an absent tool — a wiring change, not a second implementation (§11.4.227). NOT DONE this round: that file sits outside the working brief's declared file scope, so the gap is named and the item stays open rather than the scope being exceeded (§11.4.6). EVIDENCE. docs/qa/BOB-087/seam-coverage-measurement.md.

BOB-088 — RD2-21: Complete/verify README Tracked-Items + Status Documents table row-completeness (GA-07 remainder)

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-21, P1] Complete/verify the README Tracked-Items + Status Documents table row-completeness (GA-07 remaining half). Not independently re-verified this round whether every mandated doc (CONTINUATION.md, Issues.md, Fixed.md, PORTING-FROM-LAVA.md, both new GA/RD2 audit docs, every Status.md/Status_Summary.md pair) actually has a row. Priority: P1.

BOB-090 — RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack

Status: In progress **Type:** Task **Severity:** Medium

RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack

[BOB-136 adoption audit 2026-08-21 -> In progress] a9366c9 bumped submodules/helixqa to HelixDevelopment/qa@00c5ca4 adding banks/boba-start-sh-reload.yaml (3 cases, all three subcommands). The bank EXISTS but has never been EXECUTED: the commit states honestly that the helixqa binary was not built in-session (§11.4.3 SKIP feature_disabled_by_config) and 'live bin/helixqa list demo is deferred', with only YAML-parse evidence captured. This item's acceptance is end-to-end execution against the real compose stack — artifact-class evidence cannot close a runtime-class acceptance (§11.4.226).

BOB-093 — RD2-28: Live compose bring-up + verify rutracker ReDoS regex bounds deployed to container (closes runtime half of GA-12)

Status: In progress **Type:** Task **Severity:** High

RD2-28: Live compose bring-up + verify rutracker ReDoS regex bounds deployed to container (closes runtime half of GA-12)

[BOB-136 adoption audit 2026-08-21 -> In progress] 1c0389a proved 3 of the 4 sub-steps with strong runtime evidence: the bounded regex is confirmed in the DEPLOYED /config/qBittorrent/nova3/engines/rutracker.py inside the running container (sha256 76a6bd2e..), with a self-validated challenge + §11.4.115 RED (mutated unsafe form FAILs) and GREEN verdict JSON. The 4th sub-step is NOT met: 'capture timing of a large rutracker result page (<2s)' was not exercised — in docs/qa/BOB-093/live_search_smoke.txt rutracker returned status=empty, results_count=0 in 164ms, so no large page was ever timed.

BOB-094 — RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection

Status: In progress **Type:** Task **Severity:** Medium

RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection

[BOB-136 adoption audit 2026-08-21 -> In progress] The suite landed (test file at 4b7b21d, evidence + RED-on-mutated-classifier capture at f3ca131) with 9 test functions: 2 stress, 3 boundary, 3 chaos (network_drop, midflight_kill, input_corruption), 1 category_map. This item explicitly requires fault injection across process-kill, network-fault AND resource-exhaustion. Only 2 of those 3 classes are present — there is no resource-exhaustion scenario in tests/stress/test_tracker_fetch_stress_chaos.py. Partial.

BOB-095 — RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py for Go-side triangle

Status: In progress **Type:** Task **Severity:** Medium

RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py for Go-side triangle

[BOB-136 adoption audit 2026-08-21 -> In progress] d5c8c3e authored qBitTorrent-go/internal/service/sse_broker_stress_chaos_test.go (462 lines, 6 scenarios, 2 RED captures) and found+fixed a real double-unsubscribe ‘close of closed channel’ defect. But this item names a THREE-component triangle: sse_broker.go + internal/api/hooks.go + internal/api/scheduler_api.go. Only sse_broker.go is covered — there is no stress_chaos test anywhere under qBitTorrent-go/internal/api/. 1 of 3, partial.

BOB-097 — RD2-32: Author DDoS-class coverage for exposed download-proxy/ merge endpoints (canonical impl of RD2-07)

Status: Queued **Type:** Task **Severity:** Low

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-32, P3] Author DDoS-class coverage (RD2-07) for the exposed download-proxy/merge endpoints. Priority: P3.

BOB-100 — RD2-39: Bump submodules/ jackett one commit (canonical impl of RD2-09)

Status: Queued **Type:** Task **Severity:** Low

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-39, P3] Bump submodules/jackett one commit (RD2-09). Priority: P3.

BOB-101 — GA-19/RW-09: Is –profile go parity still a release goal? (gates RW-10..13) — OPERATOR-DECISION

Status: Operator-blocked **Type:** Task **Operator-Block-Details:** WHAT: OPERATOR must decide whether Go –profile go parity remains a v1.0.0 release goal; gates downstream RW-10..13. WHY: §11.4.66 forbids autonomous decision on release scope; §11.4.122 forbids silent removal of an existing capability. UNBLOCK: [A] operator states YES — parity remains a release goal (schedule RW-10..13 work) · [B] operator states NO — mark parity Obsolete/ superseded per §11.4.90 (reason=feature-removed with operator citation, §11.4.122) WHO: Operator **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md GA-19/RW-09, OPERATOR-DECISION] Confirmed unchanged: zero time.Ticker anywhere in qBitTorrent-go, no enricher package, zero exec.Command fan-out. Is –profile go parity still a release goal? Gates RW-10..13. Surfaced only, not auto-executed, per §11.4.66. Priority: OPERATOR-DECISION.

BOB-102 — RW-05: LAN-exposure threat model — bind tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION

Status: Operator-blocked **Type:** Task **Operator-Block-Details:** WHAT: OPERATOR must decide LAN-exposure posture: bind tunnel to 127.0.0.1 (loopback-only) OR keep 0.0.0.0 (LAN-reachable, relying on post-RD2-22 complete auth coverage). WHY: This is a security-posture policy call the agent cannot make autonomously (§11.4.66 operator-decision, §11.4.101 high-blast-radius reversible-only-if-explicit). UNBLOCK: [A] operator states BIND 127.0.0.1 (loopback-only; agent will change compose/service binding + regression-test LAN unreachability) · [B] operator states KEEP 0.0.0.0 (rely on §RD2-22 completed auth coverage; agent will add a permanent §11.4.135 LAN-auth regression guard) WHO: Operator **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RW-05 ungrouped, OPERATOR-DECISION] LAN-exposure threat model — bind the tunnel 127.0.0.1 or keep 0.0.0.0 with the now-complete auth coverage (post Root Cause 3)? Priority: OPERATOR-DECISION.

BOB-104 — §11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge

Status: In progress **Type:** Task **Severity:** Medium **Created-By:** Claude

Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md, BOB-075 agent-code-reading finding, commit e6162f7): CodeGraph 1.5.0 (up from documented 0.9.9) walked into nested-.gitignore-excluded frontend/node_modules and extension/node_modules (32,260 files / 514,456 nodes vs the 2026-06-06 baseline of 509 files / 8,906 nodes) instead of honoring frontend/.gitignore + extension/.gitignore per git check-ignore -v. Author a challenge (challenges/scripts/codegraph_gitignore_honor_challenge.sh or equivalent, e.g. wrapping ‘codegraph doctor –sniff-gitignore-honor’ if that subcommand exists, else a real re-index + file-count assertion) with §11.4.115 RED_MODE polarity: RED_MODE=1 reproduces the blowup against the live nested-.gitignore tree, RED_MODE=0 asserts the resync stays within the documented baseline order of magnitude. Full evidence: docs/codegraph/Status.md lines 91-118. UPSTREAM FILED 2026-08-18: real upstream repo is github.com/colbymchenry/codegraph (npm package @colbymchenry/codegraph, confirmed via package.json + gh repo view; NOT

vasic-digital/codegraph, correcting this ledger's original assumption). Issue: https://github.com/colbymchenry/codegraph/issues/1567 (full reproduction evidence: git check-ignore -v re-verified first-hand; two bounded synthetic repro attempts up to 63 nested .gitignore files / 960 files against the actual installed v1.5.0 binary did NOT reproduce the blowup in isolation – honest negative result, root cause not pinned to a specific line in either scanning implementation). Draft PR (diagnostic only, not a behavioral fix): https://github.com/colbymchenry/codegraph/pull/1568 – adds a logDebug() call so a future report can confirm which of the two independent gitignore-respecting code paths (git-ls-files-based vs filesystem-walk fallback) actually ran. Both PRs/issue filed via SSH (Hard Stop #2) from a fork at github.com/milos85vasic/codegraph. Status: In-progress pending upstream maintainer triage/merge – boba-side closure of this item still needs the RED/GREEN challenge script (§11.4.115) authored per the original scope, independent of upstream's response.

BOB-106 — §11.4.238 followup: §11.4.84 quiescence-check helper for the unattributed auto-commit path

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md RD2-00/BOB-068 entry, docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-00): 20 bare 'Auto-commit' commits exist in this repo's git history (confirmed via git log –oneline –all –grep, e.g. 54e313f/9c8f684/743097a/de9270b/1c36777/41179c2/7c529ca) with no ATM-NNN reference and no TDD trail, landing via ordinary git pull fast-forward from a second session/host with push access to the same remotes (mismatched commit timezone vs the investigating host, per RD2-00 Update). §11.4.84 working-tree-quiescence has no mechanical guard on this path — no gate flags a commit reaching main with a bare/templated message and no ticket citation. Author a §11.4.84 quiescence-check helper (e.g. challenges/scripts/no_unattributed_autocommit_challenge.sh) that scans the commit range since the last known-good release tag and FAILs on any commit message matching a closed bare/templated pattern (e.g. ^Auto-commit$, ^sync:) with no ATM-NNN or task/PR reference, wired into scripts/pre_build_verification.sh or the §11.4.234 commit-push-all.sh entrypoint. BOB-068 (RD2-00) remains the tracking item for identifying/stopping the source; this item is specifically the new automated CHECK.

BOB-107 — §11.4.238 followup: pre-dispatch existence check for subagent task-brief source inputs

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md SCRATCH-LOSS-2026-08-18 entry, evidence: .superpowers/sdd/task-phase1a-report.md line 21): the Phase 1a subagent (a1cc331d) discovered at task start that 5 source files its brief named as required reads (curriculum_amendment_plan_v1.md, ai_curriculum_modules_27_35_extracted.md, and 3 curriculum_analysis_modules_*.md gap analyses) were absent from the session scratchpad — root cause per that subagent's own investigation: the prior producer subagent (ae59171f) hit its session rate limit (a §11.4.147(e) API-quota crash) before writing them. curriculum_amendment_plan_v1.md remains absent from the live scratchpad as of this session's re-check. No mechanical check verifies a task brief's declared input files exist and are non-empty before the downstream consumer subagent is dispatched. Author a pre-dispatch precondition helper (project-side orchestration tooling, e.g. a small script the conductor runs before Task/Agent dispatch when a brief names required input paths) that fails closed with an actionable 'missing input, respawn the producer' message rather than silently letting a downstream agent proceed on absent evidence and fabricate content unsupported by its named sources.

BOB-109 — BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix

Status: Ready for testing **Type:** Task **Severity:** Medium **Created-By:** Claude

docs/testing/test_type_matrix.md's §11.4.27 test-type audit found zero scaling-class coverage anywhere in the tree (no scaling-tagged directory, test file, or HelixQA bank distinguishes growing-dataset/tracker-count/concurrent-user scale-out from stress-under-burst). Scope at least one scaling dimension, e.g. tracker-count scale-out in merge search against challenges/helixqa-banks/boba-services.yaml's tracker set, or the qbittorrent-proxy-go –profile go swap, with a real measured baseline.

BOB-110 — BOB-074 followup: UX-class test coverage (accessibility/usability) absent

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

docs/testing/test_type_matrix.md’s §11.4.27 test-type audit found UI functional coverage (Vitest + Playwright) but nothing framed around usability/accessibility/UX outcomes specifically. Scope an axe-core or equivalent accessibility pass over the Angular frontend, covering WCAG checks, keyboard-nav coverage, and screen-reader labeling.

BOB-111 — BOB-074 followup: configure real rate limiting for boba’s 3 public HTTP endpoints

Status: In progress **Type:** Task **Severity:** High **Created-By:** Claude

Source inspection across the whole stack (2026-08-18) verified no rate-limit mechanism exists for :7185 (qBittorrent WebUI proxy), :7187 (merge search service), or :7189 (boba-jackett) – no slowapi/limiter/throttle import in download-proxy/src/, no rate-limit middleware in qBitTorrent-go/internal/middleware/ (only cors.go + logging.go) or internal/jackettapi/ (only auth/cors middleware tests, no rate middleware), and no nginx/reverse-proxy service in docker-compose.yml. Candidate remediations documented in docs/testing/ddos_resilience.md: nginx-in-container reverse proxy with limit_req_zone (most portable, adds a container); a FastAPI slowapi dependency for the merge-search service; a Gin rate-limit middleware for boba-jackett following the existing internal/middleware/ pattern. qBittorrent’s own WebUI bandwidth-shaping settings were NOT verified to cover request-rate (only bandwidth) – do not assume they close this gap without checking.

BOB-114 — BOB-074 followup: self-validation golden-bad fixture for the rate-limit detector

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

challenges/scripts/ddos_resilience_challenge.sh’s –self-validate mode currently only ships a golden-bad fixture for the crash-resistance detector (per §11.4.107(10)/§11.4.201). The rate-limiting detector has no matching golden-bad fixture proving it would actually FAIL a synthetic no-rate-limit-enforced artifact, so an unvalidated rate-limit detector could silently pass a broken/absent rate-limit deployment. Add a synthetic fixture (e.g. a stub server that never returns 429/503 under burst) and assert the detector correctly reports the absence, closing the self-validation gap for this detector class.

BOB-120 — 3rd forced-logout incident 2026-08-18 23:45:49 — SIGKILL user@1000 + preventive-timer-inside-user-slice architectural gap

Status: Queued **Type:** Bug **Severity:** Critical **Created-By:** AI

3rd forced-logout incident 2026-08-18 23:45:49 — SIGKILL user@1000 + preventive-timer-inside-user-slice architectural gap

[BOB-136 adoption audit 2026-08-21 -> DELIBERATELY LEFT QUEUED] The CM-CLOSURE-SEAM-BINDS gate classifies this row as a CONTRADICTION ‘via closes, commit d84d226’. That is a CARRIER match, not a closure. The matching text in d84d226 is ‘closing BOB-120 requires an out-of-user-scope watchdog, not more documentation’ — a statement of what closure would take, and the same commit states ‘New architectural finding, filed as BOB-120 (Critical, left Queued)’. d84d226 FILED this item; it did not close it. The architectural fix remains unlanded: BOB-121, which carries it, is still Ready for testing and its own body states it is NOT CLOSED, with two operator decisions owed and the watchdog UNTESTED AGAINST A REAL FORCED LOGOUT (survival inferred from cgroup topology, not observed). Moving this row on documentation alone would be precisely the §11.4.238 coverage-escape bluff this item’s own text warns against. Status unchanged; the gate finding is expected to persist until the out-of-scope watchdog actually lands and is verified.

BOB-121 — External watchdog for the forced-logout architectural gap (task #85, incident #3)

Status: Ready for testing **Type:** Task **Severity:** Important **Created-By:** Claude

Phase 1 design-only proposal: the BOB-116/task-77 resource-pressure preventive systemd –user timer runs inside user@1000.service, the exact pool it monitors, so it cannot fire when that pool dies (proven by incident #3, docs/qa/BOB-120/, 22:42+22:57 fires then blocked 23:45:49-23:49:00). Recommends Option B (user crontab reusing pre-existing crond.service in system.slice, no new root service) kept alongside the existing timer, with Option A (new root-owned systemd unit) as escalation path if Phase 1.5 live cron/cgroup verification is adverse. Proposal: docs/proposals/external-watchdog-for-forced-logout-architectural-gap.md. Operator decision required per §11.4.66 before any implementation – NOT implemented in this task.

Progress 2026-08-21: Flight recorder built, installed, and recording. First finding: NOTHING was watching at all — the root watchdog is LoadState=not-found and the BOB-116 user timer inactive. The blocking spike REFUTED the proposal's rationale while confirming its conclusion: this vixie-cron does invoke pam_systemd so the tick lands in session-N.scope, but that scope is a SIBLING of user@1000.service, and the real incident killed exactly one unit (73 'user@1000.service: Killing process' lines while session-18.scope merely deactivated). So the recorder does not need to survive — its SCHEDULER does, and crond in system.slice survived all seven incidents. Captures the pre-event memory/PSI/thread ramp that nothing post-hoc can recover, plus boot id, the gap itself, unit start timestamp, cgroup pids, and OOM cgroup attribution. Evidence: 5 records under real cron at exact 60s spacing; a staged analogue on a DISPOSABLE unit reaching 'VERDICT: SESSION TEARDOWN'; golden-good against the REAL BOB-120 log giving the correct diagnosis (k_unit_kill=1, k_oom=0); healthy-host quiet under load 8.15. NOT CLOSED — two operator decisions are owed: whether to keep it installed (one crontab line, zero signals, zero power verbs, 32K, reversible via uninstall.sh), and the root system.slice watchdog which is written but needs one su. UNTESTED AGAINST A REAL FORCED LOGOUT: survival is inferred from cgroup topology, not observed, and it does NOT hold against a whole-slice sweep or KillUserProcesses=yes. ## BOB-129 — Potential production slowapi/starlette defect flagged by Task 105 subagent

Status: Fixed (→ Fixed.md) **Type:** Bug **Severity:** Medium

Task #105 subagent (fixing 9 slowapi test failures) reported honestly that the same slowapi/starlette incompatibility likely hits the production /search and /search/stream endpoints under real HTTP traffic — evidence: the FastAPI TestClient (which goes through the full ASGI middleware stack like real requests do) reproduces the same isinstance() failure pattern the 9 test failures exhibited. Not yet reproduced against the running boba stack because the qbittorrent-proxy container currently exposes no host ports (running on gluetun network stack). Recommended investigation: (1) confirm defect by triggering /search kickoff

through gluetun network stack, (2) if reproduced, determine whether the fix belongs in production code (adding response: Response params) or a version pin (slowapi vs starlette compat) or a middleware refactor. §11.4.238 discovery-channel escape prevention: manual QA must NOT be the discoverer. §11.4.108 Layer 3 verification: needed on a clean deployment before any release.

BOB-131 — qbittorrent-proxy podman conmon crash — pre-existing, surfaced during BOB-129 investigation

Status: Fixed (→ Fixed.md) **Type:** Bug **Severity:** Medium

BOB-129 subagent found qbittorrent-proxy container DEAD mid-investigation (podman conmon crash). Recovery via ./start.sh –no-build worked. Pre-existing, unrelated to BOB-129 slowapi work but surfaced by it. Investigation needed: (a) how long was container dead before discovery? (b) what triggered the conmon crash? (c) is there a §11.4.144 always-follow / §11.4.128 always-record signal we should add to detect this class earlier? Post-recovery: container now unhealthy (see BOB-132). §11.4.238 discovery-channel escape: was originally found by a subagent investigating something else, not by dedicated container health monitoring.

Closed 2026-08-21 — THE PREMISE OF THIS TICKET IS FALSE, and that is the finding.

No `conmon` process crashed. This item conflated TWO UNRELATED EVENTS. Over the full 7.5-day journal retention the only conmon messages above warn are conmon REPORTING failures (`Failed to create container: exit status 1`, `Failed to write 137 to exit file`) — never crashing. A conmon crash would put conmon in a kernel segfault line; the only such line in a week is a `python3` one.

EVENT 1, once, 2026-08-20 17:56:26 CEST: `python3[314359]: segfault at 70 ... in libpython3.12.so.1.0`, with the container's own stdout ending mid-`"  File "` — the dumper died writing it. Proven at machine level rather than inferred: the kernel's `Code:` bytes at IP were byte-matched against the library INSIDE the running container (MATCH), decoding to `mov r14,[r12]` (frame->f_executable = NULL) then `mov rax,[r14+0x70]` (code->co_filename) -> fault; the preceding `lea` resolves to the literal `" File "` with edx=7, its exact length and exactly the text the log truncated after. Self-healed in 0.03s via `restart: unless-stopped`. Zero recurrences since.

EVENT 2, the 14h06m43s absence: a HOST POWER-OFF (`systemd-logind: The system will power off now!`), container exited 0. `restart: unless-stopped` does not survive a power cycle, and `boba-stack.service` is linked but disabled.

ALL THREE LOOKALIKES EXCLUDED WITH EVIDENCE: cgroup OOM-kill (oom_kill 0, zero OOM lines in 7.5 days, flight recorder k_oom=0 across all 62 samples); cgroup memory-ceiling (REAL — memory.max=805306368 with 1584 memory.events in 48 min — but that is reclaim, not a kill, and cannot produce SIGSEGV); §12.12 thread exhaustion (ulimit -u 65536, peak 1559 = 2.4%, no EAGAIN / 'failed to create new OS thread' anywhere).

THE ACTIONABLE RESIDUAL IS SPLIT OUT AS BOB-157 (High): the crash vector is our OWN diagnostic — `download-proxy/src/main.py:135` calling `faulthandler.dump_traceback(all_threads=True)` — hitting upstream python/cpython#116008 / #128400, fixed and backported to 3.13/3.14 with NO 3.12 backport, on a container shipping 3.12.13. Still armed.

DELIVERED: `docs/guides/container-death-triage.md` (a 6-class decision table plus the three confusions above), `docs/incidents/2026-08-21-bob131-container-death-triage.md`, and `scripts/diagnostics/bob131_container_death_triage.sh` (TDD RED 7/8 -> GREEN 8/8, deterministic 5/5, both §1.1 mutations FAIL — collapsing oom_kill/ceiling breaks 5 fixtures including the negative control). The investigating agent caught its OWN selftest passing by race (an `awk '…; exit'` SIGPIPE) and fixed it before reporting. ## BOB-135 — Test isolation: test_list_hooks_after_create fails in bulk suite (Permission denied /config)

Status: Ready for testing **Type:** Bug **Severity:** Low

Task #109 subagent found: tests/unit/test_merge_api_route_contracts.py::TestHooksEndpoint::test_list_hooks_after_create fails when run in bulk suite order with 'ERROR api.hooks:hooks.py:102 Failed to save hooks: [Errno 13] Permission denied: /config'. Passes in isolation (2.02s clean). Root cause: full-suite ordering pollution — some earlier test leaves state that makes hooks try to write to /config (which the test env doesn't own). Pre-existing, unrelated to BOB-126/BOB-129 chain. Fix strategy: identify the polluting test, add teardown or use a proper tempdir fixture for hooks storage in the offending test.

BOB-137 — Merge service on 7187 wedges while the same process still serves 7186 (GIL starvation by one spinning thread)

Status: In progress **Type:** Bug **Severity:** High **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-20T14:46:52Z **Reported-By:** Claude

What (the report, verbatim): The merge service on port 7187 wedges after hours of uptime: the port stays bound and the container keeps reporting “healthy”, but every HTTP request hangs until the client times out. Measured 2026-08-20 16:41 UTC+2 on a container up 3h53m:

curl http://localhost:7186/ -> HTTP 200 in 0.096s (proxy, same process) curl http://localhost:7187/ -> HTTP 000 after 6.0s (merge service, WEDGED)

Both ports are served by the SAME process (pid 2330069, fd 3 = 7186, fd 7 = 7187), so this is not a crashed worker – one loop inside a live process has stopped servicing requests while another in the same process is fine.

Thread state at the time of the wedge (/proc/2330069/task, 16 threads): - tid 2330529: state R, wchan 0 <- ONE thread spinning on CPU - tid 2330528: state S, wchan do_sys_poll <- the healthy 7186 poll loop - the other 14: state S, wchan futex_wait_queue

That is the GIL-starvation signature: a thread busy-looping in Python holds the GIL and every other thread queues on the GIL futex. 7186 survives because do_sys_poll releases the GIL; the 7187 async loop needs sustained GIL time to service a request and starves. Process CPU was 20.6% while idle.

Corroborating evidence: seven sockets held by the process sit in CLOSE-WAIT with unread bytes still in the receive queue (Recv-Q 162, 162, 162, 79, 1, 1, 1) – clients sent a request and hung up, and the app never read it or closed the fd. The listener had also accumulated an unaccepted backlog (Recv-Q 6 on the 7187 LISTEN socket) at first observation. The container log’s last line is 2h before the observation, so the service processed nothing in that window.

NOT fd exhaustion: only 48 of 16384 fds were open.

ROOT CAUSE NOT ESTABLISHED. All four `while True` loops in the service (routes.py:166, search.py:1169, streaming.py:161, streaming.py:359) are correctly bounded and awaited, so the spin

is not a naive unslept loop. py-spy could not attach to name the spinning frame – the host has kernel.yama.ptrace_scope=1, which denies non-child attach. Per the §11.4.102 Iron Law no fix is proposed until the spinning frame is identified.

Next diagnostic step: obtain a Python stack. Either run py-spy INSIDE the container (musl wheel), or set ptrace_scope=0 for the duration of one dump, or add a SIGQUIT/faulthandler.register() dump hook to main.py so the running service can be asked for its own stacks without ptrace.

DISCOVERY CHANNEL (§11.4.238): found by an agent probing the host by hand while investigating an unrelated test-suite result – NOT by automated QA. This is a coverage escape in its own right; see the sibling item on the health check that was structurally incapable of observing it.

Affected scope / file-scope manifest: download-proxy/src/main.py, download-proxy/src/merge_service/, download-proxy/src/api/streaming.py, docker-compose.yml (qbittorrent-proxy)

Reproduction / context: Leave the qbittorrent-proxy container running for several hours with search traffic (a full tests/security run is sufficient). Then: curl –max-time 6 http://localhost:7186/ returns 200; curl –max-time 6 http://localhost:7187/ returns 000. Confirm with: ss -ltnp | grep 7187 (unaccepted backlog) and awk '{print $3}' /proc//task/*/stat (one R thread, rest futex_wait_queue).

Acceptance criteria: The spinning frame is identified from a real Python stack dump (not inferred), the busy-loop is fixed at its root, and a regression guard proves 7187 still answers after a sustained-traffic soak. Evidence: before/after curl timings on both ports plus a thread-state census showing no permanently-R thread.

[BOB-136 adoption audit 2026-08-21 -> In progress] 1dd7b0a ESTABLISHED the root cause with captured evidence (16 stack dumps showing Deduplicator.merge_results() called synchronously at search.py:914 on the event-loop thread; loop thread sustained 81-98% user-space CPU while all other threads showed d_utime=0). The remediation landed under BOB-145 (0572b71, now Fixed), whose own text is headed 'BOB-145 - the 7187 wedge' and which refuted the assumed O(N^2) cause by profiling. BOB-137 and BOB-145 therefore describe the SAME defect; per §11.4.214 that is a link/dedup decision, not a unilateral close, and no post-fix re-observation of the multi-hour wedge is recorded against BOB-137 itself. Left open pending that linkage decision.

Live verification 2026-08-21 — REDUCED BUT NOT ELIMINATED. Deliberately NOT closed.

The service was confirmed running POST-FIX bytes before any measurement, six independent ways: served sha256 == committed == worktree across 20 merge-service/api/main files; fix markers 10 inside the container vs 0 at the pre-fix commit; `.pyc` header decode showing embedded source mtime+size matching the actual file (so the import reused it and the loaded module was compiled from exactly this source); fix markers present in the LOADED bytecode; and the process starting 823s AFTER the fix hit disk. A first attempt at this comparison reported routes.py as stale — an INSTRUMENT ARTIFACT (marshal back-reference encoding differs between a fresh compile and a pyc load), caught before it became a false positive.

Soak, same script and concurrency, precondition reached (11,340 results over 516 tracker responses / 12 searches ~ 945 per merge, larger than BOB-145's N=800 maximum):

```
PRE-FIX  (quoted from the recorded report): 22 of 26 probes dead
on 7187 (84.6%)
POST-FIX (measured):                          2 of 141 dead (1.4%)
```

The dead-count alone understates the residual: 25.5% of probes stalled >1s (<0.1s 65.2% / 0.1-1s 9.2% / 1-5s 18.4% / >=5s 5.7% / dead 1.4%). Both dead events show the exact BOB-137 asymmetry — 7186 answering in 0.077s while 7187 timed out at 10s — and both coincide with the loop thread sampled at state R with wchan=0, the GIL-starvation signature (tid independently confirmed as the loop thread by its idle wchan do_epoll_wait).

WHY THIS ROW STAYS OPEN: the acceptance evidence this item names is '22/26 dead -> 0/N dead'. Achieved: 22/26 -> 2/141. The user-visible symptom — 7187 unresponsive while 7186 answers in the same process — STILL OCCURS, twice in 15 minutes. That is precisely BOB-145's own predicted residual: search.py:914 is still a plain synchronous call, and removing the symptom needs a change at that CALL SITE (offload or await), which BOB-145 explicitly scoped out.

Two criteria that ARE satisfied: no permanently-R thread (48 of 80 census samples had zero R threads), and the in-process 20s watchdog logged 0 stalls — with a control needle, since that same watchdog produced 167,971 bytes of dumps pre-fix. ## BOB-141 — CLAUDE.md claims the Go profile serves 7186/7187/7188 but its container binds only 7187 — doc contradicts the Dockerfile

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-20T14:56:38Z **Reported-By:** Claude

What (the report, verbatim): CLAUDE.md's Architecture section states:

“qbittorrent-proxy-go (Go/Gin, opt-in via –profile go) – replaces the Python proxy on 7186, 7187, 7188”

The container cannot deliver that. Read from source rather than prose:

qBitTorrent-go/Dockerfile:16 CMD [“/app/qbittorrent-proxy”] (ONE binary) qBitTorrent-go/Dockerfile:15 EXPOSE 7187 7188 (declares 2) cmd/qbittorrent-proxy/main.go r.Run(fmt.Sprintf(“:%d”, cfg.ServerPort)) (binds 1) internal/config/config.go:58 ServerPort = MERGE_SERVICE_PORT (default 7187)

So the qbittorrent-proxy-go container binds 7187 ONLY. webui-bridge is a SEPARATE binary (cmd/webui-bridge, /bridge/health, cfg.BridgePort) that this container never starts, and nothing in it binds 7186 either – although the compose service does set PROXY_PORT=7186 and BRIDGE_PORT=7188 in its environment, which reinforces the wrong impression. EXPOSE likewise declares a port nothing binds.

WHY THIS MATTERS BEYOND TIDINESS: this prose was used as the source of truth when first authoring config/served_ports.yaml, producing a manifest entry of [7186, 7187, 7188] for that service. The healthcheck gate built on it then FAILED a service whose healthcheck was already correct – a §11.4.201(1) false-positive refusal caused directly by trusting the doc over the Dockerfile. The manifest was corrected against source; the doc was not, and will mislead the next reader the same way.

Either the doc is wrong, or the Go container is under-provisioned relative to intent (it should also run webui-bridge and a 7186 listener). Determining WHICH is part of this task – do not simply reword the doc to match the current binary if the intended design was a three-port container.

Related: the Go service’s compose block sets PROXY_PORT and BRIDGE_PORT that no process in the container consumes, and EXPOSE lists 7188 unbound. Whichever way the above resolves, those should agree with reality afterwards.

Affected scope / file-scope manifest: CLAUDE.md (Architecture + Port Map), docker-compose.yml (qbittorrent-proxy-go env/EXPOSE), qBitTorrent-go/Dockerfile

Reproduction / context: Read qBitTorrent-go/Dockerfile:16 (single CMD) against CLAUDE.md’s ‘replaces the Python proxy on 7186, 7187, 7188’; confirm cfg.ServerPort resolves to MERGE_SERVICE_PORT=7187 in internal/config/config.go:58.

Acceptance criteria: Doc and container agree, with the direction of the fix decided deliberately (correct the doc, or provision the container to match the documented intent). PROXY_PORT/ BRIDGE_PORT env and EXPOSE lines agree with what the container actually binds.

BOB-143 — Orphaned .worktrees/ dirs (46M, unresolvable gitdir) pollute gate scan scope and manufacture false BOB-126-class findings

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-20T15:08:18Z **Reported-By:** Claude

What (the report, verbatim): `.worktrees/ci-split-workflows/` (13M) and `.worktrees/completion-initiative-phase-0/` (33M) are ORPHANED: `git worktree list` reports only the main checkout, so neither is a registered worktree. Each contains a `.git` POINTER FILE whose target gitdir no longer exists, so git cannot resolve HEAD, branch, or status inside them – every query returns empty.

They are gitignored (.gitignore:122), so nothing tracks them and nothing will ever notice them drifting.

WHY THIS IS NOT COSMETIC: they pollute the scan scope of whole-tree gates and manufacture false findings. Measured 2026-08-20 by the §11.4.32 sweep:

- `cm_test_mock_pid_explicit_int` reported 2 violations at tests/ unit/merge_service/test_deadline_tunable.py:44 – BOTH inside these orphaned trees. The MAIN tree's copy of that file is already hardened (it sets `mock.pid = 12345` and patches os.killpg/os.getpgid) and passes the gate cleanly. The finding read as a live §11.4.263 / BOB-126-class defect and was not one.
- 6 of the 57 "missing anchor carrier" files flagged by the propagation gates were likewise `.worktrees/**`.

That is the §11.4.201(1) false-positive shape sourced from scan scope, and it costs real investigation time: a reader triaging "2 live BOB-126 violations" reasonably treats it as a host-safety emergency.

CLARIFICATION (established during triage, so the record is not alarming): these trees are NOT a kill(-1) vector. Their `download-proxy/src/merge_service/ search.py` contains ZERO `os.killpg` calls – they predate that cleanup code entirely – so there is no unguarded

signal call to reach. Additionally `pyproject.toml` sets `testpaths = ["tests"]`, so a plain `pytest` run does not collect from `.worktrees/`. No host-safety risk was found. The defect is scan-scope noise plus 46M of unreferenced disk.

WHY REMOVAL IS NOT DONE AUTONOMOUSLY (§11.4.122 / §11.4.124 / §11.4.101): because git cannot resolve their HEAD, it is NOT possible to prove their contents are merged into main. Deleting unprovable-provenance work is exactly the irreversible, operator-owned decision §11.4.122 reserves. Two options for the operator: (a) confirm removal (they are stale dev scratch dirs) – reversible only from backup, so a §9.2 pre-op backup should precede it; or (b) keep them and add `.worktrees/` to the gate scan-scope exclusion list as a §11.4.224(E)-fenced, checked-in, justified entry.

Either way the exclusion list is the cheaper immediate mitigation and does not destroy anything.

Affected scope / file-scope manifest: .worktrees/ci-split-workflows/, .worktrees/completion-initiative-phase-0/, gate scan-scope config

Reproduction / context: git worktree list shows only the main checkout; git -C .worktrees/

> log -1 returns empty (gitdir target missing). Run the §11.4.32 sweep and observe cm_test_mock_pid_explicit_int report 2 violations, both under .worktrees/, while the main-tree file passes the same gate.
>
> **Acceptance criteria:** Whole-tree gates no longer report findings sourced from orphaned worktrees: either the dirs are removed after operator confirmation with a §9.2 pre-op backup, or .worktrees/ is added to a checked-in §11.4.224(E)-fenced exclusion list with justification. Verify by re-running the sweep and confirming zero .worktrees-sourced findings.

BOB-144 — /theme/stream calls the disconnect probe unguarded — fail-closed but via an uncaught traceback, inconsistent with the two SSE generators

Status: Fixed (→ Fixed.md) **Type:** Bug **Severity:** Low **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-20T15:53:57Z **Reported-By:** Claude

What (the report, verbatim): `/theme/stream` in download-proxy/src/api/routes.py:167 calls the disconnect probe with NO guard at all:

```
while True:
    if await request.is_disconnected():
        break
```

If that probe raises, the generator dies with an UNCAUGHT exception.

IMPORTANT — this is NOT the BOB-139 fail-open. The effect here is fail-CLOSED: the stream stops, and the enclosing `finally: store.unsubscribe(queue)` still runs, so the subscriber queue is released and nothing leaks. The outcome is CORRECT; the manner is not.

What is wrong with it: - it terminates via an uncaught traceback rather than a clean SSE `close` event, so the client sees a truncated stream instead of a reason; - the failure is not logged as a probe failure, so a systematically raising probe would show up as recurring tracebacks with no diagnosis; - it is inconsistent with the two SSE generators in streaming.py, which after BOB-139 emit `event: close` with reason `disconnect_probe_failed` and log a warning. Three call sites of the same probe now behave two different ways.

The BOB-139 fix deliberately did not touch this file (ownership boundary, §11.4.119), and flagged it honestly rather than fixing it out of scope.

Acceptance: /theme/stream uses the same probe-failure discipline as the streaming.py generators — a clean close event with the `disconnect_probe_failed` reason plus a logged warning — proven in BOTH directions (§11.4.201(1)): a raising probe closes the stream cleanly AND a normally-connected client still streams uninterrupted. Prefer reusing the shared helper introduced by BOB-139 rather than a third copy of the logic (§11.4.251).

Honest boundary (§11.4.6): the production probe-failure rate is UNKNOWN — nobody has measured how often `is_disconnected()` actually raises. This is filed on the inconsistency and the missing diagnosis, not on a measured incident rate.

Affected scope / file-scope manifest: download-proxy/src/api/routes.py (~line 167, stream_theme)

Reproduction / context: Monkeypatch Request.is_disconnected to raise, open /theme/stream, observe the generator dies with an uncaught traceback rather than emitting event: close with reason disconnect_probe_failed as streaming.py now does.

Acceptance criteria: Same probe-failure discipline as streaming.py (clean close + disconnect_probe_failed reason + logged warning), verified both directions, reusing the BOB-139 helper rather than a third copy.

BOB-145 — Fix the 7187 wedge: offload and/or memoise Deduplicator.merge_results so O(N^2) regex work stops blocking the asyncio event loop

Status: Fixed (→ Fixed.md) **Type:** Bug **Severity:** High **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-20T16:01:43Z **Reported-By:** Claude

What (the report, verbatim): BOB-137 established the root cause: Deduplicator.merge_results() is called as a PLAIN SYNCHRONOUS CALL at merge_service/search.py:914 and therefore executes ON the asyncio event-loop thread. While it runs, uvicorn's only loop runs no callback — no accept, no read, no write — so port 7187 stops answering entirely. Measured episodes of 9m37s and 5m56s, self-clearing, recurring under ordinary traffic.

This item is the FIX. BOB-137 is the diagnosis and stays separate per the §11.4.102 Iron Law — the investigation deliberately applied no fix.

Three candidate directions, not yet chosen:

1. OFFLOAD — hand merge_results to a thread executor (asyncio.to_thread / run_in_executor). Smallest change, immediately unblocks the loop. Does NOT make the work cheaper, so a large enough merge still burns a core; and it introduces concurrency around self._last_merged_results and metadata, which must be checked for races before it is called safe.

2. MEMOISE — _normalize_name is called at FOUR sites per comparison, each running 5 re.sub, and _extract_identity_from_result twice more with a ~15-

re.search chain. lru_cache has ZERO matches in that module today, so the seed's own normalisation is recomputed for every candidate. Caching the per-result normalisation is a large constant-factor win with no concurrency risk.

3. REDUCE THE COMPARISON SET — the O(N^2) shape itself (blocking/bucketing by a cheap key before pairwise comparison). Largest win, largest change, highest risk of altering dedup RESULTS — which would need its own correctness evidence, not just a speed measurement.

4. then (a) is the likely order: (b) is risk-free and may alone bring the merge under the threshold, and (a) guarantees the loop is never blocked regardless.

MANDATORY for whoever takes this: - RED FIRST (§11.4.43/ §11.4.224): a test that FAILS on the current code by demonstrating the loop is blocked during a merge — e.g. assert a concurrent request to 7187 is served within a bounded time while a large merge runs. A pure speed benchmark is NOT the RED test; the defect is loop-blocking, not slowness. - BOTH POLARITIES (§11.4.201(1)): the loop stays responsive under a large merge AND dedup results are unchanged for the existing corpus. A fix that speeds up merging while changing which duplicates are detected is a different defect. - Use the existing instrument: scripts/ diagnostics/bob137_soak.sh reproduced the wedge 22/26; it is the natural GREEN check. - Honest boundary carried from BOB-137: measured growth is SUPER-LINEAR but not fully quadratic at N<=400 (2.4-3.4x per doubling vs 4.0). Worst case is quadratic BY CODE STRUCTURE. Do not cite "measured N^2".

Affected scope / file-scope manifest: download-proxy/src/ merge_service/search.py:914, download-proxy/src/ merge_service/deduplicator.py

Reproduction / context: bash scripts/diagnostics/ bob137_soak.sh — reproduced the wedge in 22/26 probes (7187 dead, 7186 alive throughout).

Acceptance criteria: Port 7187 answers within a bounded time while a large merge runs (loop never blocked), AND dedup results are unchanged for the existing corpus. Proven with the soak repro flipping from 22/26-dead to 0/N-dead, plus a dedup-equivalence check.

BOB-146 — Constitution §11.4.252 detector undercounts by 29% (30 vs 42 AST ground truth) — 4 distinct blind spots make its output a floor, not a census

Status: Fixed (→ Fixed.md) **Type:** Bug **Severity:** High **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-20T16:10:50Z **Reported-By:** Claude

What (the report, verbatim): The constitution's §11.4.252 detector (constitution/scripts/gates/ cm_dangerous_combination_fail_closed.sh) UNDERCOUNTS by 29%. Measured against an independent AST instrument (structure, not text — a valid control needle per §11.4.201(7)) on boba's plugins tree:

```
gate reports        : 30
AST ground truth    : 42
missed              : 12
false positives     : 0
```

A gate that misses 12 of 42 while reporting a confident number is a §11.4.201(6) false-null: its output reads as a census when it is a FLOOR. Anyone fencing an exclusion list against that number (§11.4.224(E)) would write the fence against an undercount and lock the invisible sites out of scope permanently.

FOUR DISTINCT CAUSES, each independently falsifiable:

L1 — TRAILING COMMENT DEFEATS THE REGEX (10 of the 12). The pattern anchors the handler line with …: `[[:space:]]*$`, so `except Exception:  # noqa: S110` never matches. The irony is load-bearing: the sites a human consciously reviewed and annotated are exactly the ones the gate cannot see.

L2 — TUPLE CLAUSE DEFEATS THE REGEX (2 of the 12). The exception-type group is `[A-Za-z_.]+`, which cannot match `(`, so `except (OSError, ValueError):` is invisible.

L3 — A COMMENT BETWEEN `except` AND `pass` DEFEATS THE BODY CHECK. The detector reads exactly `lineno+1`. Zero current instances, but demonstrated live. This one has a nasty second-order effect: a well-intentioned reviewer adding an explanatory comment INSIDE a handler makes that site vanish from the gate. The triage agent hit this itself — its

first patch put comments inside two handlers, and the resulting count would have read 28 while only 6 sites were genuinely eliminated. It caught and corrected that rather than reporting the better number, which is exactly the §11.4 discipline working.

L4 — THE SHAPE ITSELF, not the regex. The body must be exactly `pass`, so `except: return <default>` is out of scope BY CONSTRUCTION. 13 such sites remain in boba (12 vendored `community/`, 1 `linuxtracker.py:51`), plus `plugins/rutor.py:121` (`except: return  int(time.time())`) which a structural invariant found and which is now fixed. Returning a silent default is the SAME defect class as `pass` — arguably worse, because the caller receives a plausible value rather than nothing.

RECOMMENDED FIX: replace the text-matching detector with an AST-based one for Python. `except` handlers are trivially enumerable from `ast.Try.handlers`, and a handler whose body neither re-raises nor logs nor returns a distinguishable failure signal is decidable structurally. Text matching cannot reach L1-L4 without accumulating epicycles; each of the four above is a separate regex patch under the current design.

WHATEVER THE FIX, IT MUST BE FALSIFIABLE (§1.1): the paired mutation must include one fixture per cause — trailing comment, tuple clause, comment-before-pass, and `return <default>` — each of which the CURRENT detector passes and a correct one must FAIL. A negative control is required too: a correctly-narrowed handler that logs and re-raises must NOT fire (§11.4.201(1)).

CONSUMER-SIDE NOTE (already fixed, boba): pre_build_verification.sh invariant 39 counted the gate's own SUMMARY line as a finding, because that line also starts with the failure marker. Each failing root added exactly one phantom, so the reported total read 38 when the gate itself said 36. Fixed by matching the finding STRUCTURE (a finding names " at :"; a summary never does) rather than the marker glyph. That is a separate defect from the four above, in the consumer, and is NOT part of this item.

Affected scope / file-scope manifest: constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh (+ its paired mutation test)

Reproduction / context: Run the gate over plugins/ and compare to an AST enumeration of ast.Try.handlers: gate=30, AST=42, 12 missed, 0 false positives. Each cause reproduces standalone: except Exception: # noqa (L1); except (OSError, ValueError): (L2); a comment between except and pass (L3); except: return

(L4).

Acceptance criteria: Detector finds all 42 AST-confirmed sites with zero false positives, with a paired §1.1 mutation carrying one fixture per cause (all four currently PASS the detector and must FAIL a correct one) plus a negative control that must NOT fire on a correctly-narrowed logging-and-re-raising handler.

BOB-148 — Standing red unit test nothing tracked: test_no_credentials asserts has_session False, gets True — real defect or non-hermetic test, undecided

Status: Fixed (→ Fixed.md) **Type:** Bug **Severity:** Medium **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-20T16:28:57Z **Reported-By:** Claude

What (the report, verbatim): tests/unit/test_auth_coverage.py:303 TestAllTrackersAuthStatus::test_no_credentials

```
assert result["trackers"]["qbittorrent"]["has_session"] is False
E   assert True is False
```

The test asserts that with NO credentials configured, qbittorrent reports has_session=False. It reports True.

PROVEN PRE-EXISTING, by experiment rather than by reasoning: a detached worktree at the session-start commit e335dde reproduces the identical failure. Nothing in this session touched download-proxy/src/api/auth.py or this test file (git log over the session range for both paths is empty). It was already red and nothing was tracking it — which is the actual defect worth recording: a standing red unit test that no item names will be re-discovered forever and attributed to whoever touches the tree next. It was in fact attributed twice today before being pinned down.

TWO HYPOTHESES, both plausible, NEITHER confirmed (§11.4.6 — do not pick one without evidence):

(H1) A REAL DEFECT: the auth-status path reports has_session=True on the strength of something other than a credential — a cached cookie, a default-constructed client, or a truthy default in the status assembler. If so, the operator-visible consequence is that the dashboard would show a tracker as authenticated when it is not.

(H2) A NON-HERMETIC UNIT TEST (§11.4.27(A)): the test reads real state rather than a stub, and the live qBittorrent container at :7185 — which is UP and healthy on this host — supplies a genuine session. Under that hypothesis the test would PASS on a host with the stack stopped, which makes it environment-dependent, i.e. FLAKY, and §11.4.248 quarantine territory rather than a product bug.

DECISIVE EXPERIMENT (cheap, and it distinguishes them in one run): execute this single test with the stack stopped, or with the qBittorrent host/port pointed at a closed port. If it PASSES, H2 holds and the fix is to make the unit test hermetic (mock the client) — the product is fine. If it still FAILS, H1 holds and the fix is in the auth-status assembly path.

Do NOT stop the stack casually to run this — other work depends on it. Prefer pointing the test at an unbound port via env override, which is reversible and affects nothing else.

NOTE the §11.4.226 evidence-class consequence: if H2 holds, this test has been asserting a RUNTIME condition from a unit-test layer all along, which is why it reads red on a developer host and would read green in a clean CI container — the worst possible polarity, since the environment that most resembles production is the one where the test stays silent.

Affected scope / file-scope manifest: tests/unit/test_auth_coverage.py:303, download-proxy/src/api/auth.py (auth-status assembly)

Reproduction / context: timeout 300 .venv/bin/python -m pytest tests/unit/test_auth_coverage.py::TestAllTrackersAuthStatus::test_no_c

redentials -q –import-mode=importlib -> assert True is False. Reproduces identically in a detached worktree at e335dde (session start).

Acceptance criteria: The H1/H2 experiment is run and recorded. If H2: the unit test is made hermetic (no live-stack dependency) and passes with the stack both up and down. If H1: the auth-status path no longer reports has_session without a credential, with a RED test capturing it first.

BOB-149 — Managed-plugin count diverges 43/42/48 across constitution, CLAUDE.md and the README badge; the badge is hand-maintained and unguarded

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-20T22:35:42Z **Reported-By:** Claude

What (the report, verbatim): The managed-plugin count diverges three ways across the repo:

.specify/memory/constitution.md : 43 (and enumerates all 43 by name) CLAUDE.md : 42 (“42 managed plugins”) README.md badge : 48 (plugins-48)

AUTHORITATIVE SOURCE, per the constitution’s own text (“the install-plugin.sh managed list is the canonical curated set”): the PLUGINS=() array in install-plugin.sh holds exactly 43 entries.

Counted with a control needle (§11.4.201(7)(b)): the extractor was verified to match a known member (“rutracker”) before its count was trusted. A first attempt returned 0 because the array entries are QUOTED (“academictorrents”) and the pattern was unquoted — a false-null caught by the needle rather than reported as “no plugins”.

So the constitution is CORRECT and the other two are the drifted copies.

Neither wrong number is harmless: - CLAUDE.md's 42 is the file agents read as project instruction, so the wrong number is the one most likely to be propagated into new work. - README.md's 48 matches NEITHER the curated array (43) NOR plugins/*.py (36) NOR plugins/**/*.py (69). It is not a stale-but-once-true number; nothing in the repo currently equals 48, so its provenance is unknown.

The badge is additionally UNGUARDED: compute-badges.sh does not derive the plugins badge at all — it is one of the hand-maintained ones. That is the same shape as the challenges/pre-build badges fixed at a6f36fa, which had been stale by 7 and 14 while the script printed "cross-checked, matches existing badge". The plugins badge escaped that fix because it was never in the script's scope.

Acceptance: 1. CLAUDE.md states 43, matching the authoritative array. 2. The README plugins badge is DERIVED by compute-badges.sh from the PLUGINS=() array (not hand-typed), so it cannot silently drift again. 3. tests/unit/test_compute_badges_all_badges_updated.sh is extended to cover the plugins badge, so the guard covers every machine-derived badge rather than the subset that happened to be fixed first. 4. Verified in both directions (§11.4.201(1)): the guard FAILs when the badge disagrees with the array, and PASSes when they agree.

Filed rather than fixed inside the v1.4.0 constitution amendment so the documentation fix and the governance change stay independently reviewable.

Affected scope / file-scope manifest: CLAUDE.md, README.md (plugins badge), scripts/compute-badges.sh, tests/unit/test_compute_badges_all_badges_updated.sh

Reproduction / context: Count the PLUGINS=() array in install-plugin.sh (43, needle-verified) and compare against CLAUDE.md ('42 managed plugins') and the README badge (plugins-48).

Acceptance criteria: CLAUDE.md says 43; the README badge is derived by compute-badges.sh from the array; the badge guard covers it; both polarities verified.

BOB-150 — pre_build_verification.sh invariant labels read N/44 but only 35 invariants are labelled

Status: Queued **Type:** Task **Severity:** Low **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-21T14:46:07Z **Reported-By:** AI

What (the report, verbatim): The pre-build runner announces each invariant as [N/44], but only 35 invariants carry a label and only 35 distinct CM-* gate names are announced. Numbers 33-38, 40, 42 and 43 are unused; there are no duplicates. An operator reading the output sees '[44/44]' scroll past and reasonably concludes 44 invariants ran, when 35 did - the output overstates coverage by 9. This is PRE-EXISTING and was surfaced while wiring CM-OWNERSHIP-INVARIANTS, which deliberately took free slot 33 precisely to avoid a 35-label renumbering that would have conflicted with concurrent work. That gate neither introduced nor fixed this. Filing rather than absorbing it silently, per the closed-or-tracked rule: an unstated finding reads as no finding.

Affected scope / file-scope manifest: scripts/pre_build_verification.sh

Reproduction / context: grep -oE '[[0-9]+/44]' scripts/pre_build_verification.sh | wc -l -> 35 (denominator says 44); numbers 33-38, 40, 42, 43 are unused, no duplicates. Distinct CM-* names announced in the file: 35, which agrees with the label count and not with the denominator.

Acceptance criteria: Either the denominator matches the real number of labelled invariants, or the numbering is compacted to be contiguous - and whichever is chosen, a gate or the runner itself derives the denominator rather than restating it as a literal, so the two cannot drift apart again.

BOB-151 — CM-SCRIPT-DOCS-SYNC is a named gate with no implementation, and 24 of 36 scripts have no companion doc

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-21T15:05:33Z **Reported-By:** AI

What (the report, verbatim): The §11.4.18 script-documentation rule names a gate CM-SCRIPT-DOCS-SYNC, but no executable file in this repository implements it. Because nothing measures the obligation, 24 of 36 scripts under scripts/ have no docs/scripts/.md companion - two thirds of them. This is the named-gate ledger gap the constitution itself warns about: a gate that is named but never implemented reads as coverage while providing none, and the corpus grows words instead of enforcement. Surfaced while writing the two ownership companions, which were themselves only written because the feature plan asked for them explicitly - not because any gate demanded it. Filing rather than absorbing: an unstated finding reads as no finding, and a 67% documentation gap that nothing reports will not shrink on its own.

Affected scope / file-scope manifest: scripts/*.sh, docs/scripts/*.md, scripts/pre_build_verification.sh

Reproduction / context: grep -rl CM-SCRIPT-DOCS-SYNC –include='*.sh' –include='*.py' . (excluding submodules) -> 0 files. Control needle in the same command shape: CM-OWNERSHIP-INVARIANTS -> 3 files, CM-KILLPG-PGID-GUARD -> 7 files, so the search is not blind. Then: for s in scripts/*.sh; do [-f docs/scripts/$(basename $s .sh).md] || echo missing; done -> 24 of 36 missing.

Acceptance criteria: Either the gate is implemented and the 24 missing companions are written (the count becoming a monotone-decreasing ratchet so it cannot grow), or the obligation is explicitly scoped down to a defined subset with the reason recorded. What is NOT acceptable is the current state, where the rule is stated and nothing measures it.

BOB-152 — Constitution sweep walks vendored third-party code: 82% of one gate’s 38,291 findings come from submodules/

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-21T15:10:28Z **Reported-By:** AI

What (the report, verbatim): The constitution sweep passes –root at the repository root, so every gate walks vendored third-party code the project neither wrote nor ships: submodules/helixqa/tools/opensource/perfetto, chroma, skyvern, mem0 and so on. Measured today, 82% of one gate’s findings originate there. This is a false-positive refusal at scale: it fails the sweep over code that cannot be fixed here, and it buries the 4497 first-party findings that might actually matter under 31496 that do not. A second, subtler instance: cm_killpg_pgid_guard flags its OWN golden-bad fixtures and the BOB-126 incident docstrings - the gate reporting on the very artifacts that prove it works. Surfaced while fixing the .worktrees/ half of this problem (BOB-143), which was the smaller 6% slice; that fix was deliberately scoped to .worktrees/ rather than quietly widened to cover this, because excluding submodules/ is a materially larger decision about what the sweep is for and deserves its own review.

Affected scope / file-scope manifest: scripts/verify-all-constitution-rules.sh, config/constitution-sweep.conf, constitution/scripts/gates/*

Reproduction / context: config/constitution-sweep.conf line 27 passes ‘DEFAULT –root @ROOT@’, so every gate walks the whole repository. Per-tree census of cm_oracle_strategy_named_and_independent over the full root: 38291 findings total - 31496 (82%) from submodules/, 2280 (6%) from .worktrees/, 4497 from the real tree. cm_killpg_pgid_guard: 18 findings - 9 from submodules/, and of the 8 in the real tree several are the gate’s OWN golden-bad fixtures and BOB-126 docstrings.

Acceptance criteria: The sweep scans code this project actually ships. Vendored third-party trees under submodules/ are excluded or scoped explicitly, and any gate’s own golden-bad fixtures are excluded from its own scan - with the exclusion validated in BOTH directions (a planted violation in first-party code must still FAIL) so this does not become narrow-until-green.

BOB-153 — Go profile cannot build: go.mod requires go 1.26.2 but the Dockerfile builder is golang:1.23-alpine

Status: Fixed (→ Fixed.md) **Type:** Bug **Severity:** Medium **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-21T15:40:00Z **Reported-By:** AI

What (the report, verbatim): The Go backend profile is unbuildable. qBitTorrent-go/go.mod declares ‘go 1.26.2’ while qBitTorrent-go/Dockerfile builds with ‘FROM golang:1.23-alpine’, so the build aborts before compiling anything. Found while attempting feature 002 quickstart Scenario 6, which exercises the go profile to confirm the ownership fix covers every service (FR-016) - the scenario could not run at all, for a reason unrelated to ownership. Two consequences worth separating. First, this is a plain build defect and is pre-existing. Second, and more awkward, it means FR-016 coverage for the go profile currently rests on surface-equivalent measurement (its Dockerfile’s final stage is alpine:3.19 with no USER directive, so it runs as container root and inherits the correct rootless mapping) rather than on a live running service. That reasoning is sound but it is not the same evidence as a probe against a real container, and it is recorded as the weaker evidence it is. Also noted while investigating: the go profile has no Hard-Stop-#3-compliant invocation path - start.sh has no –profile flag, so the only route is a raw compose command, and it would collide on port 7187 with the running Python proxy since both use network_mode host.

Affected scope / file-scope manifest: qBitTorrent-go/go.mod, qBitTorrent-go/Dockerfile

Reproduction / context: grep ‘^go’ qBitTorrent-go/go.mod -> ‘go 1.26.2’; grep ‘FROM golang’ qBitTorrent-go/Dockerfile -> ‘FROM golang:1.23-alpine AS builder’. Building the go profile fails: ‘go: go.mod requires go >= 1.26.2 (running go 1.23.12; GOTOOLCHAIN=local)’.

Acceptance criteria: The go profile builds. Either the builder image is raised to a toolchain satisfying go.mod, or go.mod’s directive is lowered to what the builder provides - and whichever is chosen, a check ties the two together so they cannot drift apart again, because nothing currently compares them.

BOB-154 — Host venv and production container run different starlette versions (1.4.1 vs 1.6.0)

Status: Ready for testing **Type:** Bug **Severity:** Medium **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-21T15:46:07Z **Reported-By:** AI

What (the report, verbatim): The host virtualenv used to run this project’s unit tests resolves starlette 1.4.1 while the running qbittorrent-proxy container resolves 1.6.0. Every other implicated package matches, so this is a genuine unpinned-transitive-dependency drift rather than a deliberate difference. It matters because it silently weakens every test result: a green suite on the host is evidence about 1.4.1, and production is 1.6.0. A behavioural change between those versions would be invisible to the tests that exist to catch it, which is the build-once-run-the-same-bytes property the constitution asks for. Surfaced while investigating BOB-129, where the defect happened to reproduce IDENTICALLY at both versions - which is what allowed that ticket’s slowapi/starlette-incompatibility premise to be refuted. That was luck, not design: the next divergence may not be version-independent, and nothing would tell us.

Affected scope / file-scope manifest: download-proxy/requirements.txt, .venv, container qbittorrent-proxy

Reproduction / context: .venv/bin/python -c 'import starlette;print(starlette.__version__)' -> 1.4.1 ; podman exec qbittorrent-proxy python -c 'import starlette;print(starlette.__version__)' -> 1.6.0. Same fastapi (0.141.1), same slowapi (0.1.10), same limits (5.8.0) - starlette alone diverges.

Acceptance criteria: The interpreter that runs the tests and the interpreter that serves production resolve the same versions, pinned so they cannot drift apart silently - or, if a divergence is deliberate, it is declared and a check asserts the declared pair rather than leaving it to chance.

BOB-155 — workable-items diff reports 'DB and Markdown are in sync' having opened zero Markdown files when –issues/–fixed are omitted

Status: Fixed (→ Fixed.md) **Type:** Bug **Severity:** High **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-21T15:57:00Z **Reported-By:** AI

What (the report, verbatim): The flagless form of the sync checker is a false-null generator. Called without –issues/–fixed it prints the same reassuring 'DB and Markdown are in sync' it prints after a real successful comparison, while having read no Markdown whatsoever. A blind instrument and a genuinely clean tree return the identical quiet verdict, which is precisely the failure class the constitution's measurement-integrity rules exist to prevent - and here it lives inside the project's own sync-verification tool. This bit for real today: the flagless form was used to VERIFY a reconciliation of five tracker rows and reported 'in sync'. The reconciliation happened to be correct - re-checked afterwards with the path-ful form, which is clean on the current tree and correctly reports 2 differences against a planted divergence - so the conclusion was right and the evidence for it was worthless. Nobody would have noticed, because the output is indistinguishable from a real pass. Surfaced by the BOB-136 investigation. Not fixed there because the file lives in the constitution submodule, which carries its own review and commit discipline and was dirty with concurrent work at the time. The caller-side exposure

was closed instead (a gate now asserts zero flagless callers), but the engine itself still ships the trap for every other consumer of that submodule.

Affected scope / file-scope manifest: constitution/scripts/workable-items/cmd/workable-items/sync.go

Reproduction / context: Plant a real divergence: change a **Status:** line in docs/Issues.md only. Then: workable-items diff –db docs/workable_items.db -> ‘diff: DB and Markdown are in sync’ (WRONG - it compared nothing). The same command WITH paths: workable-items diff –db docs/workable_items.db –issues docs/Issues.md –fixed docs/Fixed.md -> ‘2 difference(s)’ (correct). Restored byte-identical after the test.

Acceptance criteria: Omitting –issues/–fixed either REFUSES with a non-zero exit naming the missing input, or defaults to the conventional paths and says which files it read. What must never happen again is a confident ‘in sync’ verdict from a comparison that opened no Markdown at all - the verdict must always name its inputs so a reader can tell a real check from a vacuous one.

Closed 2026-08-21 — constitution commit `16b67b0`, pushed to 8 upstreams. Chose REFUSE plus an explicit `--db-only` opt-in, because the alternative (defaulting to conventional paths) is NOT implementable in a shared submodule without baking one consumer’s `docs/Issues.md` layout into it. Sibling precedent settled it: `sync md-to-db` and `sync db-to-md` in the same file ALREADY refuse when every path flag is absent, so `diff` was the exception. Every verdict now NAMES ITS INPUTS — “compared 152 Markdown item(s) against 152 DB item(s); read

/Issues.md,

/Fixed.md” — which is the property that makes the failure class impossible rather than merely unlikely.

ROOT CAUSE WITH HISTORY: this defect was INTRODUCED BY THE FIX FOR ITS OWN MIRROR IMAGE. Before 2026-08-10 the flagless form ran the absent-in-Markdown loop against an EMPTY parsed set and flagged every DB row — a FAIL-bluff. The fix added a `haveMarkdown` gate that correctly silenced the noise, then fell through to the unconditional success verdict. Same seam, opposite polarity:

a FAIL-bluff traded for a PASS-bluff. Worth recording, because “we fixed the false positives” is exactly how a false negative gets installed.

TWO ADJACENT DEFECTS SURFACED, both worse than the one filed: (1) BOB-155 was ALREADY being caught by a test’s GREEN branch — a standing red nobody saw because the suite is evidently never run in GREEN mode, a coverage escape where the check existed and nothing executed it; (2) the suite was RED IN BOTH POLARITY MODES beforehand, and one test’s GREEN branch had begun ASSERTING THE DEFECT. Both reconciled to assert the new mechanism rather than fake-passed or reverted. One lesser instance fixed in passing: `md-to-db` printed the hardcoded label `Issues.md:` even when reading a renamed tracker — a verdict misnaming its input AND a baked-in filename in a shared submodule.

Siblings audited EMPIRICALLY, not assumed: md-to-db and db-to-md refuse; `validate` has no optional inputs and names its item count, so it cannot be blind. Commit seam verified against a purpose-built PRE-FIX binary and the fixed one — identical behaviour, because all three of its checks already pass –issues/–fixed. Zero project literals among added lines, control-needled. ## BOB-156 — BOB-145 event-loop regression guard is load-sensitive and flaky: 8786ms under host load vs a 900-1500ms ceiling calibrated on a quiet host

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-21T17:01:20Z **Reported-By:** AI

What (the report, verbatim): The regression guard added with the BOB-145 fix asserts an event-loop block ceiling calibrated on a quiet host. Under real host contention the block window scales with BOTH N and load, so the same code that passes at 349ms median can measure 8786ms - worse than the pre-fix number the test exists to detect. That makes it FLAKY, and a flaky test is corrosive in a specific way this project has already recorded: every ignored red trains everyone to dismiss the next one, so a real regression eventually gets waved through as ‘that one again’. Two directions are wrong: raising the ceiling until it stops failing would blind it to the defect, and leaving it flaky poisons every future run. Found while verifying BOB-137 against the live service.

Affected scope / file-scope manifest: tests/unit/merge_service/test_dedup_event_loop_blocking.py

Reproduction / context: Under host load 18-24 on 8 cores (concurrent agents), the same N=400 merge froze the event loop for 8786ms - WORSE than the 3970ms pre-fix figure the ceiling was calibrated against. 2 fail / 2 pass across four runs.

Acceptance criteria: The guard gives the same verdict on a loaded host as on a quiet one - or it measures something contention-independent. A threshold that only holds when nothing else is running is not a regression guard, it is a weather report.

BOB-157 — Our own BOB-137 stall watchdog can segfault the merge service: faulthandler dump_traceback(all_threads=True) hits an unpatched CPython 3.12 defect

Status: Fixed (→ Fixed.md) **Type:** Bug **Severity:** High **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-21T17:18:18Z **Reported-By:** AI

What (the report, verbatim): Our own diagnostic is a crash vector, and it is still armed. download-proxy/src/main.py:135 calls faulthandler.dump_traceback(file=sink, all_threads=True) on a live 18-thread process; three further registrations at 214/217/219 use all_threads=True as well. Upstream python/cpython#116008 and #128400 are the same NULL f_executable in dump_frame() at Python/traceback.c:1190 - FIXED and backported to 3.13/3.14, with NO 3.12 backport listed. The container ships Python 3.12.13. Container logs still show 'BOB-137 stall watchdog armed: stall>20.0s'. It has not fired again only because BOB-137's root cause was improved enough that the loop rarely stalls past the threshold - but BOB-137 is NOT closed, and its live verification the same day measured the loop still blocking, with 18.4% of probes stalled 1-5s and two dead events in 15

minutes. So this is latent, not resolved: one 20s stall away. Note the perverse shape - the worse the wedge gets, the more likely the tool built to diagnose it is to kill the process, destroying the evidence it exists to capture. Found while investigating BOB-131, whose own premise (a conmon crash) turned out to be false: no conmon process crashed; the ticket conflated this python3 segfault with an unrelated 14-hour absence caused by a host power-off.

Affected scope / file-scope manifest: download-proxy/src/main.py:135 (and the registrations at 214/217/219)

Reproduction / context: 2026-08-20 17:56:26 CEST, one occurrence: kernel ‘python3[314359]: segfault at 70 … in libpython3.12.so.1.0’ plus the container’s own truncated dump ending mid-’ File ‘. The kernel Code: bytes at IP were byte-matched against the library inside the running container (MATCH), decoding to mov r14,[r12] (frame->f_executable = NULL) then mov rax,[r14+0x70] (code->co_filename) -> fault; the preceding lea resolves to the literal’ File ’ with edx=7, its exact length. The watchdog had fired 17 times in 16 minutes that day; dump 17 completed to the file sink, then the stderr pass crashed.

Acceptance criteria: The diagnostic cannot crash the service it diagnoses. Either all_threads=True is dropped for the periodic dump, or the dump is gated behind something that cannot fault on a live multi-threaded process, or the runtime moves to a Python where the upstream fix is present - and whichever is chosen, the choice is recorded against the upstream issue so a later runtime bump does not silently re-arm it.

BOB-158 — tests/conftest.py cannot run on the production interpreter: binds asyncio.events._get_event_loop_policy, a 3.13+ private API, while production is 3.12.13

Status: Queued **Type:** Bug **Severity:** High **Created-By:** AI

Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T17:29:10Z **Reported-By:** AI

What (the report, verbatim): The test suite has silently acquired a dependency on an interpreter production does not run. tests/conftest.py:207 calls asyncio.events._get_event_loop_policy() - a PRIVATE API that does not exist before Python 3.13 - in an AUTOUSE fixture, so it affects every test. The host venv is CPython 3.14.6; the container is 3.12.13, pinned deliberately in docker-compose.yml as python:3.12-alpine and independently declared twice more in pyproject.toml (ruff target-version py312, mypy python_version 3.12). So the venv contradicts the project's own declared target by documentary evidence, not opinion. THIS IS NOT A THEORETICAL GAP: every green suite run to date is evidence about 3.14.6 while users are served 3.12.13, and this specific incompatibility means the suite CANNOT run on the production interpreter at all - it produces 870 teardown errors. It went unnoticed because nobody had ever run the suite on 3.12. Found while fixing BOB-154 (dependency drift), whose reconciliation is BLOCKED on this: rebuilding the venv on 3.12 is the fix for the drift, and doing so makes the suite unrunnable until this is resolved. A candidate patch shape was verified in isolation on BOTH 3.12.13 and 3.14.6 under -W error::DeprecationWarning, but not as an integrated change - and another stream was editing conftest.py concurrently, so it was deliberately not applied.

Affected scope / file-scope manifest: tests/conftest.py:207 (_cleanup_event_loop fixture)

Reproduction / context: .venv/bin/python -c 'import asyncio.events as e; print(hasattr(e,"_get_event_loop_policy"))' -> True (3.14.6). podman exec qbittorrent-proxy python -c same -> False (3.12.13). Running the suite on 3.12.13 produces 870 teardown errors. A second incompatibility in the same fixture: policy.get_event_loop() raises DeprecationWarning on 3.12, which this project's pytest config turns into an error.

Acceptance criteria: The suite runs on the interpreter production runs. Both incompatibilities in _cleanup_event_loop are resolved, verified on 3.12.13 under -W error::DeprecationWarning, and the venv is rebuilt on 3.12 so every subsequent test result is evidence about the deployed runtime.

BOB-159 — Warm ./start.sh over an already-running stack leaves the FR-004d repair window open

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** T041 independent review (IMPORTANT-2), partially remediated

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-21T19:01:17Z **Reported-By:** T041 independent review (IMPORTANT-2), partially remediated

What (the report, verbatim): The –recreate path was fixed this round: run_ownership_precondition -> stack_down -> run_ownership_repair -> stack_up, so the repair walks a tree no container can write to. That ordering is now asserted behaviourally by three new checks in tests/unit/test_start_reload_recreate.sh, each killed by its paired 1.1 mutation (11/11 RED-OK).

The WARM path is NOT covered by that fix and this item tracks the remainder. On a warm ./start.sh the gate runs before THIS invocation brings anything up, but if the stack is ALREADY running, containers write while the repair walks. A container can then create a new non-operator-owned file BEHIND the walk, after which the completion marker records complete over a tree that is not, and those stragglers are never repaired without a manual –force.

BOUNDED, not dismissed: after any successful pass the marker is valid for the scope fingerprint and the repair short-circuits without walking (marker_is_valid), so the window requires a stale-or-absent marker AND a live stack together. Declaring a new scope entry re-arms the marker, which is exactly how that combination arises in practice.

NOT DONE THIS ROUND, with the reason stated rather than hidden: the clean guard is a cheap marker-only probe mode on scripts/ownership_repair.sh that answers has-this-scope-been-repaired without walking the tree. The existing –dry-run cannot serve: it deliberately SKIPS the marker fast-path (FORCE=0 && DRY_RUN=0 guard) and always walks, so using it as a warm-start probe would walk the tree twice on every start. Adding that mode requires editing ownership_repair.sh, which another stream was actively

editing during this round; duplicating marker_is_valid into start.sh instead would be a near-identical fork (11.4.251). Deferred to this item rather than done unilaterally or silently.

The misleading comment at the warm-start call site (which claimed the gate runs before any container writes) has been corrected to state this boundary honestly.

Affected scope / file-scope manifest: start.sh (warm-start dispatch, run_ownership_gate call site); scripts/ownership_repair.sh (marker fast-path)

Reproduction / context: 1. Ensure the stack is UP. 2. Change config/owned_paths.yaml so the scope fingerprint changes (this re-arms the marker by design, data-model E2). 3. Run a warm ./start.sh (no –recreate). 4. The ownership repair walks and chowns the declared tree while containers are still running and able to write into it.

Acceptance criteria: A warm ./start.sh cannot complete an ownership repair while a container that writes to a declared location is running. Either the repair is deferred with an actionable refusal naming –recreate, or the stack is quiesced for the walk. Asserted behaviourally in tests/unit/test_start_reload_recreate.sh alongside the existing PRECONDITION_BEFORE_DOWN / REPAIR_AFTER_DOWN / REPAIR_BEFORE_UP checks, each with a paired 1.1 mutation that kills it.

BOB-160 — tests/pre_build/ and tests/ownership/ ran by nothing — closed by extending invariant 30 + wiring ci.sh

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-21T19:02:48Z **Reported-By:** AI

What (the report, verbatim): Independent §11.4.209 review (IMPORTANT-3) found two of the feature’s strongest automated checks were never executed by any runner: (1)

tests/ownership/test_container_writes_owned_files.py — the §11.4.115 RED-turned-regression-guard for FR-011/FR-007, proven via a docker-compose.yml revert mutation — was moved out of tests/integration/ (commit 58d340a, to escape an autouse fixture hang) but no runner (ci.sh, test.sh, run-all-tests.sh) was ever extended to cover tests/ownership/. (2) tests/pre_build/test_*.sh, the §1.1 paired-mutation meta-tests for the scripts/pre_build/check_cm_*.sh gate family, was executed by NOTHING — self-reported honestly in commit 04742d7's own message ('STILL OPEN (reported, not fixed): tests/pre_build/ is executed by NOTHING') but never tracked as a workable item (a §11.4.197 loss-of-requirements gap) and never wired into scripts/pre_build_verification.sh invariant 30, which globbed only tests/unit/test_*.sh.

Affected scope / file-scope manifest: ci.sh; scripts/pre_build_verification.sh invariant 30 (CM-BASH-UNIT-TESTS-EXECUTED)

Reproduction / context: grep -rn test_container_writes_owned_files ci.sh test.sh run-all-tests.sh scripts/ returned zero runner hits (only docs/specs mentioned the path); grep in scripts/pre_build_verification.sh showed invariant 30's for-loop globbing only tests/unit/test_*.sh, never tests/pre_build/test_*.sh.

Acceptance criteria: ci.sh gains a runtime-gated pytest tests/ownership/ stage (skips honestly, rc=0, when no podman/docker present); scripts/pre_build_verification.sh invariant 30's glob covers both tests/unit/test_*.sh and tests/pre_build/test_*.sh, verified by an extracted standalone run of the invariant's exact block reporting a non-zero RAN count that includes files from both directories.

BOB-161 — The §11.4.69 CM-NO-FAIL-OPEN-SKIP gate is mandated but does not exist in this project

Status: Queued **Type:** Task **Severity:** High **Created-By:** BOB-092 remediation, residual finding

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-21T19:31:12Z **Reported-By:** BOB-092 remediation, residual finding

What (the report, verbatim): §11.4.69 names CM-NO-FAIL-OPEN-SKIP as one of three mandatory pre-build gates: it audits sink-side probe helpers and FAILs if any code path converts an empty or unreachable response into a PASS-counting SKIP for a feature class with a sink-side probe. This project does not have it.

Why this matters now rather than in the abstract: the BOB-092 remediation just removed TWO fail-opens of exactly this class from tests/e2e/test_live_stack_evidence.py — the nnmclub SKIP-on-404 (already removed at 7baef2b) and a surviving sibling at test_iptorrents_is_authenticated_in_search that skipped on 'not authenticated and status != success' while asserting 'treating as transient outage'. That assertion was false for rejected credentials (upstream_http_403) and for a broken container (plugin_env_missing), both of which are definitive product failures the product already classifies via error_type.

Two fail-opens of one class, found one at a time, is the §11.4.146 extend-to-all-cases signal and the §11.4.238 coverage-escape signal together: the regime did not surface either, an agent reading code did.

The remediation's own guards are AST-structural and lethal under three discriminating mutations, but they LIVE IN THE FILE THEY GUARD, so deleting that file evades them entirely. A pre-build gate is the durable home because it audits the corpus rather than one file.

Honest boundary (§11.4.6): this item does NOT claim the two remediated fail-opens are unguarded — they are guarded, with runtime evidence (13 passed in 97.36s against the live stack). It claims the CLASS has no corpus-wide detector, so the next instance in a different file is invisible again.

Affected scope / file-scope manifest: scripts/pre_build/ (gate absent); tests/e2e/test_live_stack_evidence.py (guards currently live inside the file they guard)

Reproduction / context: grep -rl 'CM-NO-FAIL-OPEN-SKIP' scripts/ tests/ returns exactly ONE hit, and it is tests/e2e/test_live_stack_evidence.py — the file the guards live in, not a gate. Control needle: the same query for CM-OWNERSHIP-INVARIANTS (a gate that does exist) returns 3 files, so the instrument can see gate tokens and the single hit is a real absence, not a blind zero (§11.4.201(7)(b)).

Acceptance criteria: A scripts/pre_build/ gate named CM-NO-FAIL-OPEN-SKIP exists, is wired into scripts/pre_build_verification.sh, and audits sink-side probe helpers for code paths converting an empty/unreachable/error response into a PASS-counting SKIP for a feature class that HAS a sink-side probe. It ships a golden-TRUE fixture (a real fail-open -> gate FIRES) and a golden-FALSE-with-carrier (an honest topology/geo skip, and a comment merely MENTIONING the phrase -> gate MUST NOT fire), per §11.4.107(10)/§11.4.201(1). Paired §1.1 mutation makes the gate FAIL before the gate is trusted.

BOB-162 — Two new guards exist but no seam invokes them — plus the brownfield adoption decision the commit guard needs

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** BOB-106/BOB-107 remediation, residual

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-21T19:35:02Z **Reported-By:** BOB-106/BOB-107 remediation, residual

What (the report, verbatim): The TESTS for both guards are now executed: pre-build invariant 30’s glob was extended from tests/unit + tests/pre_build to also cover tests/hooks. MEASURED before/after: the glob expanded to 27 suites with 0 under tests/hooks while 3 existed there; it now expands to 30 with 3 under tests/hooks. All three pass.

But the GUARDS themselves are still invoked by nothing.

check-brief-inputs.sh is honestly conductor-run rather than automatic: a brief’s required-input list lives in free-form prose that the Agent tool’s structured input does not expose, so an automatic prompt-scraping gate would itself be an unproven pattern-match. Its seam is the dispatch procedure, and that is a documentation + habit change, not a hook.

unattributed-commit-guard.sh is different and is why this item exists. Run against real history it names 14 violating commits since the last tag (and 21 bare Auto-commit subjects

exist across all refs). Wiring it into the pre-build or commit seam AS-IS would therefore refuse every commit immediately, on pre-existing debt nobody can fix in the moment.

That is precisely the 11.4.224(E) brownfield-adoption shape, and 11.4.66/11.4.122 put the choice with the operator, not with an agent inventing a ratchet. The options, stated so the decision is a decision and not a default: (a) immediate hard floor – the seam refuses until all 14 are attributed; (b) one-time monotone-decrease ratchet (the 11.4.135 pattern) – snapshot 14 as the baseline, the count may fall and may never rise, so day one is green and the disease cannot spread; (c) changed-commits-only – gate only commits created from now on, with a scheduled deadline for the historical 14; (d) report-only for a stated period, then escalate.

Recommendation, with the reasoning rather than just the pick: (b). It is the pattern this constitution already uses for exactly this situation, it makes the debt visible and bounded immediately, and it cannot be satisfied by deleting the guard's name (11.4.227 closes that gaming channel). But it is the operator's call and this item is not closed until that answer is recorded.

Honest boundary (11.4.6): this item does NOT claim the 14 commits are harmful in themselves – it claims their ATTRIBUTION is absent and that nothing currently prevents the 15th.

Affected scope / file-scope manifest: scripts/hooks/unattributed-commit-guard.sh; scripts/hooks/check-brief-inputs.sh; scripts/pre_build_verification.sh; scripts/commit-push-all.sh

Reproduction / context: Both guards run correctly by hand and are covered by passing tests (9/9 and 15/15, each proven non-bluff against a no-op stub). Neither is invoked by scripts/pre_build_verification.sh or scripts/commit-push-all.sh, so neither exerts standing detection pressure (11.4.226: a guard with no execution seam is not coverage; registration is not coverage).

Acceptance criteria: Each guard is invoked by a named seam, OR carries a registered deferral pointing at this item. For unattributed-commit-guard.sh specifically, the operator has answered the adoption question below and the answer is recorded as consumer DATA – never an invented ratchet.

BOB-163 — Now that :7187 really rate-limits, the DDoS challenge’s cross-endpoint isolation assertion reads a sibling 429 as endpoint-degraded

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** BOB-114 remediation, pre-existing defect surfaced by BOB-111 landing a real limiter

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-21T19:41:08Z **Reported-By:** BOB-114 remediation, pre-existing defect surfaced by BOB-111 landing a real limiter

What (the report, verbatim): PRE-EXISTING, NOT INTRODUCED – and proven so rather than asserted: git diff shows assertion (c) is BYTE-IDENTICAL to HEAD, and the failure reproduces against the unmodified HEAD script. What changed is not the challenge but the SYSTEM: BOB-111 appears at least partially delivered, because :7187 now returns real 429s where it previously returned none. :7185 and :7189 still produce zero 429s.

This is a 11.4.248-class corrosion risk and that is why it is filed rather than left as a footnote: run_all_challenges.sh will now fail INTERMITTENTLY, and an intermittent red trains everyone to re-run until green, at which point a real regression is dismissed as ‘probably the flaky rate-limit one’.

THE DECISION, which is an operator call under 11.4.66 and which I deliberately did NOT invent:

Does a 429 from a WORKING limiter count as the sibling endpoint being RESPONSIVE?

1. YES – a 429 proves the endpoint is alive and correctly protecting itself. Assertion (c) accepts 2xx OR 429, and only a connection failure / 5xx / timeout counts as degraded. Risk: a genuinely wedged endpoint that happens to answer 429 would pass.
2. NO – keep requiring 2xx, but make the challenge limiter-aware: drain or wait out the window before the sibling probe (retry-after is served, so the budget is knowable rather than guessed).
3. Probe siblings on a path the limiter exempts (a healthz-class route), so the isolation question is asked without spending the bucket.

Recommendation with reasoning, not just a pick: (a) composed with a bounded liveness check – a 429 carrying a well-formed Retry-After IS evidence of a live, correctly-behaving service, and (b) makes the challenge slower and couples it to a window value that will drift. But the semantics of 'responsive' here are a product judgement, so the answer is recorded, not assumed.

RELATED, stated as fact rather than folded in: the detector counts 429 only, not 503, though the script header says '429 (or equivalent)'. Counting 503 would collide with the crash detector's 5xx tally. Worth deciding when BOB-111 lands limiters on :7185 and :7189.

Affected scope / file-scope manifest: challenges/scripts/ddos_resilience_challenge.sh assertion (c) cross-endpoint isolation; run_all_challenges.sh which invokes it

Reproduction / context: Assertion (c) requires a sibling-endpoint probe to answer ^2 (a 2xx). :7187 now enforces a real limiter (measured live: x-ratelimit-limit: 120, x-ratelimit-remaining: 86, retry-after: 45, server: uvicorn). A full challenge run sends roughly 250 requests to :7187, so sibling probes fired during the OTHER endpoints' tiers land on an exhausted bucket and receive 429 – which assertion (c) scores as 'endpoint degraded'. Timing-dependent: one run measured PASS=5 FAIL=2; the UNMODIFIED HEAD script against the same live stack ~30s later measured PASS=7 FAIL=0 SKIP=2.

Acceptance criteria: The operator has answered the classification question below, the answer is recorded as consumer DATA, and assertion (c) implements it. The challenge then produces the same verdict across 3 consecutive runs against a rate-limited stack (11.4.50

deterministic consistency), and the fix ships a paired 1.1 mutation proving assertion (c) still catches a genuinely degraded sibling endpoint – narrowing it must not blind it (11.4.201(1)).

BOB-164 — Live dashboard fails WCAG AA colour contrast on 21 nodes — brand heading measures 1.43:1 against a 3:1 floor

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** BOB-110 UX-class coverage, discovered by the new axe-core suite on its first live run

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-21T19:56:53Z **Reported-By:** BOB-110 UX-class coverage, discovered by the new axe-core suite on its first live run

What (the report, verbatim): This is a REAL user-facing defect, not a test-tuning artifact, and it was found by the automated regime rather than by a human squinting at the page – which is exactly the 11.4.238 posture the project is aiming for.

The static-grep oracle would NOT have found it. Measured: the served root ships an empty pre-hydration, so any check reading the raw HTML audits a page nobody sees. The violation only exists in the hydrated DOM, which is why 11.4.170 requires a rendered oracle and forbids value-equality assertions as the proof a UI is correct.

Severity reasoning, stated rather than assumed: this is user-visible and affects the primary dashboard heading, but it degrades legibility rather than breaking function, and the surface is operator-facing rather than public. Medium, not High.

The failing test was left FAILING on purpose (11.4.238) instead of silenced or marked xfail. tests/ux/ currently reports 1 failed, 16 passed; that 1 is this defect. Anyone

reading a red UX suite should read it as this item, not as flakiness – and when this is fixed the suite goes fully green, which is the signal that it is closed.

HONEST SCOPE LIMIT (11.4.6): only the dashboard landing view was scanned. The /jackett/* sub-routes and the ng-serve-hosted :4200 route set were NOT audited – the commands to close both are recorded in docs/testing/ux_accessibility.md. So this item's 21 nodes are a floor, not a total.

Affected scope / file-scope manifest: the Angular dashboard served at http://localhost:7187/ (same compiled SPA as frontend/); production component CSS, not test files

Reproduction / context: Run tests/ux/ test_live_dashboard_accessibility.py against the running merge service. axe-core v4.13.0, scanning a real Playwright-rendered JS-hydrated DOM, reports color-contrast violations on 21 nodes. Measured pairs: .brand / h1 text #9d001e on background #3c3f41 = 1.43-1.62:1 (WCAG AA large-text floor is 3:1); tagline #808080 on #3c3f41 = 2.68:1 (body-text floor is 4.5:1).

Acceptance criteria: axe-core reports ZERO color-contrast violations against the live rendered dashboard, with the fix made in production component CSS rather than by relaxing the assertion or excluding the rule (11.4.120: reconcile to the correct mechanism, never weaken the check). The existing tests/ux/ suite is the guard and already fails today, so the RED is captured – closure requires it flipping GREEN against the live surface, which is runtime-class evidence per 11.4.226.

BOB-165 — Every documented python3 -m pytest command fails at collection: user-site rpds carries a 3.13 ABI extension under python 3.14

Status: Queued **Type:** Bug **Severity:** High **Created-By:** surfaced while capturing closure evidence for BOB-092; diagnosed rather than worked around

Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T20:00:08Z **Reported-By:** surfaced while capturing closure evidence for BOB-092; diagnosed rather than worked around

What (the report, verbatim): This is a STALE-ABI-AFTER-INTERPRETER-UPGRADE defect, not a missing package. The package is installed; its compiled half is built for the previous interpreter. That distinction matters because 'pip install rpds-py' style advice can appear to succeed while leaving the 313 artefact in place.

WHY IT WAS NOT NOTICED EARLIER, stated as fact: the paths that DO work all avoid system python3. ci.sh selects an interpreter through _select_python; the long-running suites in this session ran .venv/bin/python -m pytest and passed. So the automated regime is green while the DOCUMENTED operator command is broken – an 11.4.238-shaped gap, since the escape was found by an agent capturing evidence rather than by the regime.

WORKAROUND (measured, not theorised): PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 with the needed plugins passed explicitly (-p pytest_timeout …) avoids the schemathesis autoload that drags in jsonschema -> referencing -> rpds. That is a workaround, NOT the fix: it silences the import path rather than repairing the install, and it will surprise the next person who runs the documented command verbatim.

RELATED, and worth deciding together rather than twice: BOB-154 wants .venv rebuilt on 3.12 to match the container (which runs CPython 3.12.13 while both system and venv run 3.14.6). There are therefore THREE interpreters in play. Whoever rebuilds the venv should confirm rpds resolves for the TARGET interpreter afterwards, or this same class reappears one directory over.

HONEST BOUNDARY (11.4.6): this item does NOT claim any test is wrong or any product code is broken. The product and the venv-run suites are unaffected. What is broken is the documented entry point.

Affected scope / file-scope manifest: host user site-packages (~/.local/lib/python3/site-packages/rpds/); every CLAUDE.md-documented 'python3 -m pytest …' invocation; any tooling that uses system python3 rather than .venv/bin/python

Reproduction / context: python3 -m pytest tests/e2e/test_live_stack_evidence.py -q –import-mode=importlib -> ModuleNotFoundError: No module named 'rpds.rpds', raised during collection via the schemathesis plugin autoload -> jsonschema -> referencing._core -> rpds. MEASURED root cause: system python3 is 3.14.6, but ~/.local/lib/python3/site-packages/rpds/ ships rpds.cpython-313-x86_64-linux-gnu.so – an extension built for 3.13. rpds/**init**.py does 'from .rpds import *', and a 313-tagged .so is not importable by 3.14, so the submodule genuinely does not exist for this interpreter. The venv is CORRECT and unaffected: .venv/lib64/python3/site-packages/rpds/ ships rpds.cpython-314-x86_64-linux-gnu.so and'.venv/bin/python -c import rpds' succeeds.

Acceptance criteria: python3 -m pytest tests/unit/ -v –import-mode=importlib – the command CLAUDE.md documents – reaches collection without ModuleNotFoundError, OR CLAUDE.md is corrected to document the supported runner explicitly. Either way the documented command and the working command agree (11.4.99: a guide that misleads is the documentation-layer equivalent of a PASS-bluff).

WORKAROUND SIDE EFFECTS MEASURED 2026-08-21 — the recipe is not free, and the item previously implied it was. PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 disables ALL plugin autoload, not just the broken one, so anything the suite silently relied on must be re-enabled by hand:

- It BREAKS 5 pre-existing env-dependent tests in tests/unit/test_plugin_rutracker.py — TestConfig::test_default_mirrors, test_env_mirrors_override, test_get_env_with_default, test_get_mirrors_from_env_empty, test_get_mirrors_from_env_whitespace — which need an autoloaded env plugin. PROVEN not caused by any in-flight change: running HEAD's OWN copy of that file under identical flags produces the same 5 failures.
- It makes –timeout=60 (set in pyproject.toml) an UNKNOWN ARGUMENT unless -p pytest_timeout is passed explicitly, so pytest.mark.timeout is silently inert under the naive recipe — a test believed to be time-bounded is not.

- With autoload ON and only schemathesis disabled (-p no:schemathesis), that same file is 96/96 green.

CONSEQUENCE FOR ANYONE USING THE WORKAROUND: `-p no:schemathesis` alone is strictly better than blanket autoload-disabling where it suffices, because it removes only the broken plugin. Where the blanket form IS used, -p pytest_timeout must be added or timeouts are inert, and the 5 env-dependent failures must be recognised as workaround artifacts rather than filed as defects. That last point is the real risk: the workaround manufactures failures that look exactly like product defects.

This does not change the root cause (the venv's rpds ships a cpython-313 ABI tag CPython 3.14 cannot import) or the fix (rebuild the venv — BOB-154). It documents that the interim recipe has a blast radius, so nobody reads a workaround artifact as a regression.

BOB-166 — update –status accepts a terminal status without migrating the row, so 10 closed items sit in the open tracker while validate/diff/ closure-seam all report green

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude **Assigned-To:** Claude

WHAT. §11.4.19 requires closure migration to be ATOMIC: a resolving item moves to Fixed.md, DISAPPEARS from Issues_Summary (open-only) and APPEARS in Fixed_Summary (closed-only). Ten rows violate all three properties right now — they carry a terminal status yet current_location='Issues', so they render in docs/Issues.md and are listed in docs/Issues_Summary.md with a status that literally reads '(→ Fixed.md)', while appearing 0 times in Fixed_Summary.md. Any reader of the canonical open tracker is told these are open work.

MANIFEST (measured 2026-08-21): BOB-087 (Completed), BOB-129, BOB-131, BOB-144, BOB-145, BOB-146, BOB-148, BOB-153, BOB-155, BOB-157 (all 'Fixed (→ Fixed.md)'). Confirmed rendering: BOB-155/087/157 each grep 1 in Issues.md, 0 in Fixed.md, present in Issues_Summary.md, 0 hits in Fixed_Summary.md.

ROOT CAUSE (reproduced at runtime, not inferred). ‘close’ performs the atomic migration and REQUIRES –evidence. ‘update –status’ sets any §11.4.15 closed-set value with NO evidence and NO migration — and it knows the location, because it prints it. On a COPY of the real DB: $ workable-items update –id BOB-065 –db repro.db –status ‘Fixed (→ Fixed.md)’ update: BOB-065 updated in Issues (status=Fixed (→ Fixed.md), type=Task) $ sqlite3 repro.db ‘SELECT atm_id,status,current_location …’ BOB-065|Fixed (→ Fixed.md)|Issues So the seam that is supposed to be the ONLY closure path (close, evidence-gated per §11.4.146(D3)) has a parallel unguarded path that reaches the same status while skipping BOTH the evidence requirement and the migration.

WHY NOTHING CAUGHT IT (§11.4.238 coverage-escape audit). Three standing checks stay GREEN on a DB holding the forbidden row, verified on the poisoned copy from the repo root so no cwd artifact is involved: (1) ‘validate’ → ‘OK — 164 items, all invariants satisfied’ (it has no status↔location coherence invariant); (2) ‘diff’ → ‘in sync’ (DB and Markdown agree — on the WRONG state; agreement is not correctness); (3) CM-CLOSURE-SEAM-BINDS CHECK A → PASS (it flags NON-terminal rows whose id appears in a work commit; these rows are terminal, so they are outside its predicate by construction). This was found by reading a status tally, not by the automated regime — a §11.4.238 discovery-channel escape, which is itself a defect of equal standing to the mis-located rows.

ACCEPTANCE. (a) ‘update’ REFUSES a terminal status and names ‘close’ as the correct path, with a paired §1.1 mutation proving the refusal (removing the guard must make the mutation pass). (b) ‘validate’ grows a status↔location coherence invariant that FAILS on the forbidden state, with a golden-bad fixture and a negative control (a legitimately terminal row in Fixed must NOT fire — §11.4.201(1)). (c) The 10 existing rows are drained to Fixed with class-matched evidence per row, or, where a row’s evidence cannot be produced, honestly re-opened rather than migrated on a bare assertion. (d) Honest boundary: this closes the update-path hole and the detection gap; it does not claim every historical status write was evidence-backed.

NOT CLAIMED. No fix is implemented by this filing. The 10 rows are untouched; draining them is acceptance (c) and each needs its own evidence, not a bulk UPDATE.

BOB-167 — Two SSE routes, one rate-limit class: /search/stream carries @_rl('sse_stream') but the sibling /theme/stream carries no limiter and falls to the 120/min default

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

WHAT. download-proxy exposes two Server-Sent-Events routes. They are the same expensive class — long-lived connections that hold a worker and a generator for their lifetime — but only one is rate-limit classed:

routes.py:801 @router.get('/search/stream/{search_id}') @_rl('sse_stream') <- classed routes.py:150 @router.get('/theme/stream') async def stream_theme(…) <- NO limiter decorator

_rl(cls) resolves to limiter.limit(limit_for(cls)), so /search/stream is bound to the sse_stream class while /theme/stream falls through to the application default. Measured live on the running stack: /api/v1/theme/stream reports x-ratelimit-limit 120.

PROVENANCE + A CORRECTION WORTH KEEPING (§11.4.6). This was surfaced by the BOB-109 scaling agent, whose report framed it as: 'sse_stream_limit_decorator is defined and imported/applied nowhere … SSE falls through to the 120/minute default: 24x less protected'. That framing is WRONG and was NOT filed as given. The agent searched for one symbol NAME; the wiring uses a different mechanism (@_rl('sse_stream')), and it IS applied — to /search/stream. Verified by reading routes.py:795-806 and the _rl helper at routes.py:46-51, with a control needle confirming other *_limit_decorator symbols show real usage in api/**init**.py so the zero-hit was not a blind search. The agent's MEASUREMENT (120 on /theme/stream) was correct and is what makes this a real finding; its MECHANISM was not. Both halves are recorded so the next reader does not re-derive the same wrong cause.

WHY IT MATTERS. An unclassed SSE endpoint is the cheapest way to pin server resources: each connection is held open, and the default class permits 120/min of them. The declared sse_stream class exists precisely because this route shape needs a tighter bound than ordinary GETs.

ACCEPTANCE. (a) A decision, recorded, on whether /theme/stream belongs in the sse_stream class or genuinely warrants the default — this is a policy question, not automatically a bug to patch. (b) If it belongs in sse_stream, the decorator is applied and a test drives BOTH SSE routes and asserts each returns its INTENDED class limit from x-ratelimit-limit, so a future route added without a class is caught. (c) A guard that enumerates SSE-shaped routes and fails on any that carries no explicit rate-limit class — the general form, so the third SSE route does not repeat this. (d) Honest boundary: this does not claim 120/min is exploitable in this deployment; with network_mode host and no reverse proxy every caller shares the 127.0.0.1 bucket, which BOB-111 measured and recorded separately.

NOT CLAIMED. No change made. The limit values were read from headers, never driven to exhaustion — the limiter is per-IP and shared with concurrent agents on this host (§11.4.119).

BOB-168 — run_all_challenges.sh lists scaling_horizontal_challenge.sh which does not exist on disk, so the runner references a challenge that can never execute

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude **Assigned-To:** Claude

WHAT. scripts/run_all_challenges.sh:66 lists “scaling_horizontal_challenge.sh” in its challenge set. challenges/scripts/scaling_horizontal_challenge.sh does not exist:

```
$ sed -n '66p' scripts/run_all_challenges.sh
    "scaling_horizontal_challenge.sh"
$ ls challenges/scripts/scaling_horizontal_challenge.sh
  ls: cannot access ...: No such file or directory
```

VERIFIED independently, not taken on report — surfaced by the BOB-109 scaling agent and re-checked here by direct invocation.

WHY IT MATTERS. Whether this is cosmetic or a §11.4.201 gate-honesty defect depends entirely on how the runner treats a missing entry, and that is the first thing to determine: if it SKIPs silently, the challenge bank advertises coverage it does not have (a §11.4.266 claim-vs-reality row with no passing challenge behind it); if it FAILs, the runner is permanently red for a reason unrelated to the system under test, which trains readers to ignore it. Neither outcome is acceptable; they need different fixes.

ACCEPTANCE. (a) Determine and record the runner's actual behaviour on the missing entry by invoking it, not by reading it. (b) EITHER author the challenge, OR remove the entry — with §11.4.124 discipline: check git history for whether it once existed and was deleted, since a silently-dropped challenge is the more interesting defect. (c) If the runner silently skips missing entries, that is its own finding: a missing challenge must be loud (§11.4.3 SKIP-with-reason at minimum), never absent-and-quiet.

SEVERITY. Low as a defect, but it sits on the challenge-coverage seam, so (c) may deserve its own item.

BOB-169 — 286 of 326 exported .html docs are headless pandoc fragments with no DOCTYPE and no charset, so UTF-8 section marks and arrows render as mojibake when opened directly

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

WHAT. The §11.4.65 markdown-export mandate requires every in-scope doc to ship .html/.pdf twins. 286 of the 326 .html files under docs/ (excluding dist/ and node_modules/) are pandoc FRAGMENTS — they begin at

with no <!DOCTYPE>, no / , and critically no

. Census run 2026-08-21.

CONCRETE HARM, measured not assumed. Sample docs/ BOBA_DATABASE.html: DOCTYPE 0, charset 0, and 30 lines carrying non-ASCII — the distinct characters present are § — ’ ” ” →. With no charset declaration a browser opening the file directly (file:// or a plain static host sending no charset header) falls back to its default encoding, typically windows-1252, and renders § as Â§ and → as â†’. Those are not incidental characters in this corpus: every constitutional cross-reference in these

documents is a § literal, so the mojibake lands on the most load-bearing token in the text. Fragments also carry no viewport meta, so they do not scale on mobile.

HOW IT SURFACED. Chasing a CM-DOCS-CHAIN-ENGINE-VERIFY failure on the features-status context. docs_chain sync regenerated docs/features/Status.{html,docx,pdf} and the HTML diff was 183 insertions / 0 DELETIONS — the body was untouched and a full pandoc preamble was PREPENDED, proving the committed file had been a fragment. Its sibling docs/codegraph/Status.html already began with <!DOCTYPE>, so two

derivatives in the same docs_chain config were being produced in different modes. That mismatch is what made verify FAIL, and the gate's own message ('derived docs drift from .md sources') misattributes it: the content was not drifting from the source, the generator was inconsistent.

THE DETECTION GAP (§11.4.238). Pre-build invariant 16 CM-MARKDOWN-EXPORT-SYNC PASSED on the whole corpus — 'all in-scope docs have fresh .html/.pdf siblings'. It checks PRESENCE and FRESHNESS, never VALIDITY, so a 0-byte-preamble fragment satisfies it exactly as a well-formed

document does. This was found by reading a failing gate’s diff, not by the regime: a §11.4.238 discovery-channel escape, and the missing check is the interesting half.

ROOT CAUSE IS NOT YET PROVEN (§11.4.6). Two generators exist — scripts/generate_markdown_exports.sh (invoked via workable-items-export.sh) and the docs_chain engine — and they demonstrably disagree on standalone vs fragment mode for the same source. WHICH one emits fragments, and whether it does so always or under specific flags, is NOT established here and must not be assumed. Whoever takes this item determines

it by invoking both on one fixture and comparing, before changing either.

ACCEPTANCE. (a) Determine by invocation which generator emits fragments and under what conditions; record it. (b) Make the emitting generator produce standalone documents (charset + viewport at minimum), so the two paths agree — §11.4.251: one artifact should not have two generators that disagree. (c) Regenerate the 286 affected files. (d) Extend CM-MARKDOWN-EXPORT-SYNC (or add a sibling) to assert VALIDITY, not just presence: every exported .html declares a charset. Paired §1.1 mutation — strip the

charset from one export and the gate must FAIL. Include a negative control so a legitimately standalone doc does not fire (§11.4.201(1)). (e) Honest boundary: this is about the HTML twins only; the .pdf twins embed their own encoding and are not implicated by this measurement.

ALREADY DONE. docs/ features/Status. {html,docx,pdf} regenerated via ‘docs_chain sync features-status’ (evidence qa-results/ docs_chain/ 20260821T202805Z); verify –all now exits 0. That is 3 of the 289 files; the remaining 286 are untouched.

CORRECTION 2026-08-21 (§11.4.6) — recorded openly rather than quietly edited, per the convention BOB-136's own body establishes. An earlier revision of this item asserted: "the .pdf twins embed their own encoding and are not implicated by this measurement." That assertion is FALSE. It was written without probing a single PDF.

Probing docs/BOBA_DATABASE.pdf via pdftotext returns 'Â§ 5', 'Jackettâ€™s' and 'â†' — UTF-8 byte sequences (§ = 0xC2 0xA7) decoded as latin-1 — while the SOURCE .md at the corresponding construct is clean UTF-8. So the

corruption was introduced during export, not authored.

The PDF case is WORSE than the HTML case, not merely additional. An HTML fragment still holds correct UTF-8 BYTES on disk; a browser told the right encoding renders it correctly, so adding

fixes it with no re-render. In a PDF the mis-decoded characters are baked into the text layer as glyphs — no viewer setting recovers them, and only regeneration from clean source fixes it.

MECHANISM (hypothesis, NOT proven — §11.4.6): the PDF is plausibly rendered FROM the charset-less

HTML fragment, so the missing declaration propagates into the PDF pipeline and freezes there. Consistent with both artifacts sharing one root defect, but the pipeline was NOT traced. Establish it by invocation before relying on it.

SCOPE NOT MEASURED: exactly ONE pdf was probed. The 286-file census covered .html only. How many of the ~300 PDFs carry baked mojibake is UNKNOWN and must be COUNTED, not extrapolated from a single sample. Acceptance (e) is therefore REPLACED: it previously scoped PDFs out; it now requires the

PDF corpus to be counted and regenerated alongside the HTML.

Evidence: docs/qa/BOB-169/pdf_mojibake_correction_20260821.log

ROOT CAUSE PROVEN 2026-08-21 — acceptance (a) is SATISFIED, do not repeat it. Full evidence: docs/qa/BOB-169/root_cause_proven_20260821.md

THE GENERATOR: scripts/generate_markdown_exports.sh:57 runs `pandoc -f markdown -t html5 -o "$html" "$md" --metadata title=...` with NO –standalone/-s, so pandoc emits a body fragment with no DOCTYPE, no head, and

no . A tell that the flag was intended and lost: –metadata title= is passed on that same line, and a title can only render inside a standalone document’s

, challenge markers 0 GET /forum/tracker.php?nm=debian -> 403, 5351 B, challenge markers 1, LOGIN markers 0, trs-tr- 0

The zero login markers are the load-bearing detail: the server is not asking us to authenticate, it is refusing the request before authentication is considered. Supplying valid credentials or a fresh cookie jar therefore does NOT address this.

WHY IT MATTERS. rutracker is one of the three trackers the merge service fans a query across (with kinozal and nnmclub). A 403 on its search path means it contributes nothing to every merged result set, silently — the fan-out still ‘succeeds’, it just returns one fewer tracker’s worth of results. From a user’s seat this looks like thin results, not an outage.

IT EXPLAINS TWO PREVIOUSLY-UNEXPLAINED OBSERVATIONS. docs/qa/BOB-093/live_search_smoke.txt records rutracker returning status=empty, results_count=0 in 164ms, and the BOB-136 adoption

audit reached the same observation independently. Both recorded the symptom; neither had the cause. 164ms is far too fast for a real search across a remote forum and is exactly what an immediate 403 costs.

IT ALSO BLOCKS BOB-093's FOURTH SUB-STEP. That item requires timing a large rutracker result page under 2s. No large page has ever been obtained, and this is why. That sub-step is not merely waiting on credentials, as previously assumed — it is unmeetable until the 403 is resolved.

PROVENANCE, INCLUDING A CORRECTION I MADE MID-INVESTIGATION

(§11.4.199). An independent reviewer reported a Cloudflare challenge served to curl even with cookies. Verifying, I probed index.php, got a clean 200 with no challenge markers, and was one step from recording their claim as REFUTED. That would have been wrong for a textbook reason: index.php is not the search path, so my probe never reached the precondition, and a repro that misses the precondition proves nothing — it is not evidence of absence. Probing the actual search endpoint reproduced it immediately. The reviewer was substantially right; my probe was aimed at the

wrong endpoint. Recorded because the near-miss is the instructive part, and because the two probes TOGETHER give a sharper result than either the original claim or my near-refutation: not ‘rutracker is behind Cloudflare’ (the index is open), but ‘the SEARCH endpoint specifically is refused’.

ACCEPTANCE. (a) Determine whether the 403 is permanent policy, rate/reputation-based, or triggered by a client signature the plugin can legitimately present (user-agent, header order, TLS fingerprint) — by measurement, not assumption, and WITHOUT evasion techniques that would violate the site’s

terms. (b) Whatever the outcome, the failure must become LOUD: a tracker returning 403 must be reported as a tracker ERROR in the merge result, never folded into ‘empty’ — a silent contributor is the §11.4.201(6) false-null at the product layer, and it is what let this sit unexplained across two separate investigations. (c) A test asserting that a 403 from any tracker surfaces as an error and not as zero results, with a paired §1.1 mutation. (d) If the endpoint is genuinely unavailable to us, record that as an honest capability boundary (§11.4.112) and stop advertising rutracker as a live search source until

it is — including in the README and the merge-service docs.

HONEST BOUNDARY. Measured from ONE host, ONE time, UNAUTHENTICATED. Whether an authenticated session with a browser-like client succeeds was NOT tested and must not be assumed either way. The finding is that the current code path gets a 403; it is not a claim about what every client would get.

BOB-173 — Hook create and delete return HTTP success even when persistence fails, because _save_hooks swallows every exception — a user is told their webhook exists when it does not

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude **Assigned-To:** Claude

WHAT. download-proxy/src/api/hooks.py:96-102:

```
def _save_hooks(hooks: list[dict[str, Any]]) -> None:
    try:
        os.makedirs(os.path.dirname(HOOKS_FILE),
exist_ok=True)
        with open(HOOKS_FILE, 'w') as f:
            json.dump(hooks, f, indent=2)
    except Exception as e:
        logger.error(f'Failed to save hooks: {e}')
```

It catches EVERY exception, logs, and returns None. The signature returns None, so the caller has no channel to learn the write failed. Both call sites then report success unconditionally:

```
:152  _save_hooks(hooks)                    :174
_save_hooks(hooks)
:154  logger.info('Created hook: ...')      :175
logger.info('Deleted hook: ...')
:156  return HookResponse(hook_id=..., ...)  :176  return
{'message': 'Hook deleted', ...}
```

USER-VISIBLE CONSEQUENCE, which is why this is High and not a code-hygiene nit. A user POSTs a webhook, receives HTTP 200 and a hook_id, and the hook was never written — their automation silently never fires, and the API told them it exists. Symmetrically, a user DELETEs a hook, is told ‘Hook deleted’, the file still holds it, and it fires again after the next restart. In both directions the product reports the opposite of what happened, and the only trace is a log line nobody is watching.

THIS IS THE §11.4.252 SHAPE. The path combines two dangerous capabilities from that anchor’s taxonomy — MUTATION of a shared resource (a filesystem write) and EXTERNAL SIDE EFFECT (hooks are outbound calls the system will or will not make) — so it is required to FAIL CLOSED: verify the precondition, refuse when it cannot be satisfied, and surface the refusal. Instead it fails open, and a bare ‘except Exception:’ that only logs is the exact anti-pattern §11.4.252 enumerates. At the product layer it is also a §11.4.201(6) false-null: a successful write and a swallowed failure are indistinguishable to the caller.

PROVENANCE. Surfaced as an out-of-scope observation by the independent reviewer of the BOB-135 test-isolation work — this swallow is what turned that defect into an ‘assert 0 == 1’ mystery, because the EACCES on /config was logged and discarded while the endpoint kept returning 200. Verified here directly from source before filing (the function body and both call sites read above), not taken on report. Recorded as a §11.4.238 discovery-channel escape: found by an agent reading code during an unrelated investigation, not by the automated QA regime — the coverage gap is a defect of equal standing to the defect itself.

ACCEPTANCE. (a) _save_hooks propagates failure — raise, or return a status the callers must consume. (b) Both endpoints translate a persistence failure into an HTTP error (500-class), never a success body; a create that did not persist must not return a hook_id. (c) A test drives each endpoint with the hooks file unwritable (read-only dir or a patched open raising OSError) and asserts a non-2xx status AND that a subsequent GET does not list the phantom hook — assert on the user-observable outcome, not on the log line. (d) Paired §1.1 mutation: restore the swallow; the test must FAIL. (e) Audit the same file for sibling swallows — this is a pattern, and one instance is rarely alone. (f) Honest boundary: this does not claim the write currently fails in production; it claims that WHEN it fails the user is told the opposite, and the BOB-135 investigation shows it does fail in at least one real environment.

NOT CLAIMED. No change made. No assessment of how often the write fails in the operator’s deployment.

RECORDING A SHELL ERROR OF MY OWN (§11.4.6): the first version of this description was written with the anti-pattern snippet inside backticks in a double-quoted shell argument, so the shell ran it as command substitution and the text was replaced by nothing — the stored description read ‘and is the exact anti-pattern’. This is the SECOND time this session that backticks-inside-double-quotes has corrupted content (the first mangled a commit message). Fixed here by editing through a Python client with no shell quoting in the path. Noted because a silently-truncated defect description is exactly the kind of quiet corruption §11.4.201(7)(c) warns about — the path is part of the instrument, and it failed without erroring.

BOB-174 — A corrupt hooks file reads as zero hooks and the next create silently destroys every existing hook, while the non-atomic write manufactures the corruption

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude **Assigned-To:** Claude

WHAT. A corrupt hooks file is silently indistinguishable from “no hooks configured”, and the next create then DESTROYS every existing hook while returning HTTP 200. Three defects on one path, filed together because fixing any one alone leaves the data loss reachable.

A1 — download-proxy/src/api/hooks.py:104-112. _load_hooks wraps the read in `except Exception: logger.error(...)` and falls through to `return []`. A truncated or malformed hooks.json is therefore reported to every caller as an empty, healthy hook list. Confirmed at source.

A2 — the consequence, and the reason this is High. create_hook loads, appends, saves. Given a corrupt file that load turns into [], the save writes a one-element list over the top. Every previously-configured hook is gone. Measured by the implementing agent against the ALREADY-FIXED tree, so this survives the BOB-173 write-failure fix:

```
BEFORE  on disk : ['prod-hook-0', 'prod-hook-1', 'prod-
hook-2']
GET     reports : 200 {'hooks': [], 'count': 0}    <- claims
ZERO hooks configured
DELETE  reports : 404 {'detail': 'Hook not found'} <- for a
hook that IS in the file
POST    reports : 200 hook_id=3c8a5930-...
AFTER   on disk : ['3c8a5930-...']
```

Three prod hooks destroyed, HTTP 200 throughout, nothing surfaced to the user.

A5 — _save_hooks writes with a plain `open(path, "w")`. A crash, ENOSPC, or a kill mid-write truncates the file in place. That is precisely the corruption A1 then reads as “no hooks”

and A2 overwrites. The same codebase already has the correct pattern: theme_state.py:115-126 uses tmp-file + os.replace.

WHY THE THREE ARE ONE ITEM. A5 manufactures the corrupt file, A1 misreads it as empty, A2 destroys the contents. Fixing only A1 leaves truncation reachable; fixing only A5 leaves an existing corrupt file a data-loss trigger; fixing only A2 leaves the API lying about what is configured. The chain is the defect.

DELIBERATELY NOT FIXED UNDER BOB-173, and the reasoning is sound. The implementing agent identified all three while auditing sibling swallows as that item required, and declined to fix them in the same change because: they change GET semantics on an endpoint the frontend consumes (200 -> 5xx); they need a CORRUPT-file reproduction rather than the UNWRITABLE-file one BOB-173 built; and they need a design decision that is genuinely not obvious — a MISSING hooks file legitimately means “no hooks configured”, while a CORRUPT one does not, and today’s code cannot tell those apart. Expanding BOB-173’s scope to cover them would have meant shipping that decision unexamined.

ACCEPTANCE. (a) Distinguish MISSING from CORRUPT: a missing file remains an empty list; a corrupt one is an error, never silently []. (b) Decide and RECORD what GET does on corruption — 5xx, or 200 with an explicit degraded marker the frontend can render. This is the design decision, and it should be stated rather than inferred from whatever the patch happens to do. (c) create/delete MUST NOT overwrite a file they could not parse — refuse, do not clobber (§11.4.252: mutation plus external side effect must fail closed). (d) Make the write atomic via tmp + os.replace, reusing theme_state.py’s existing pattern rather than re-inventing it (§11.4.28). (e) Tests asserting the USER-OBSERVABLE outcome, not log lines: seed a corrupt file, assert GET does not claim zero hooks, assert a create refuses rather than destroying, and assert the file still holds the original hooks afterwards. (f) Paired §1.1 mutation per guard. (g) NEGATIVE CONTROLS (§11.4.201(1)): a MISSING file must still yield an empty list and a working create; a VALID file must behave exactly as today. A fix that makes every load fail closed by failing always is not a fix.

A3, RECORDED SEPARATELY, NOT PART OF THIS CHAIN. VALID_EVENTS and HookEventType are two sources of truth for one closed set. Verified identical today, unguarded against

drift: if they diverge a hook registers successfully and then silently never fires. One assertion pinning them would close it.

NOT CLAIMED. No change made. The BOB-173 write-failure fix is real and orthogonal — it makes a FAILED write honest; it does nothing about a SUCCESSFUL write of wrong data derived from a misread file.

BOB-175 — update –location Fixed –status can still mint a row that update's own validator rejects, because the status-location guard is one-directional

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude **Assigned-To:** Claude

WHAT. The workable-items `update` seam guards the status-location invariant in one direction only. After BOB-166, `update --status <terminal>` on an Issues-located row is refused. The mirror is still open: `update --location Fixed --status <non-terminal>` exits 0 and creates a row that `update`'s own validator immediately rejects.

Verified empirically by the independent reviewer of BOB-166:

```
update --location Fixed --status 'In progress'   -> exit 0
validate                                          -> FAILS,
naming the row (check (f), fixedLocationNonTerminalStatus)
```

So the command can still mint a state its own validate call refuses. That is the §11.4.196(F) shape at the seam layer: the invariant is CONFIGURED in validate but not ENFORCED at every write path that can violate it.

WHY IT WAS DELIBERATELY LEFT OPEN IN BOB-166, and why that was right. Three reasons, all checked rather than asserted: (1) the real corpus has ZERO instances of this direction against TEN of the direction BOB-166 closed — the asymmetry in the data matched the asymmetry in the code; (2) detective coverage ALREADY existed and demonstrably

fires, so unlike BOB-166's direction this cannot rot silently; (3) and the load-bearing one — `reopenCmd` documents and SANCTIONS a transient Fixed-plus-non-terminal state via its `--location Fixed` operator override. A naive preventive guard at the update seam would collide with that override's semantics. Closing this therefore requires a design decision about how the two interact, which is a different question from the one BOB-166 was scoped to answer, and expanding that item would have meant shipping the decision unexamined.

ACCEPTANCE. (a) Decide and RECORD how a preventive guard on this direction coexists with reopenCmd's sanctioned override — this is the actual work, and the answer should be written down rather than inferred from whatever the patch does. Candidates worth weighing: exempt the reopen path explicitly, require an override flag on `update` mirroring reopen's, or leave the direction detective-only with the rationale recorded so the asymmetry is a decision rather than an oversight. (b) If a guard lands, it reuses `terminalStatuses()` — BOB-166 established one predicate source specifically so the two directions cannot drift. (c) Paired §1.1 mutation. (d) NEGATIVE CONTROL (§11.4.201(1)), and it is the sharp one here: the sanctioned reopen path MUST still work. A guard that breaks reopenCmd's documented override is a false-positive refusal, and worse than the gap it closes, because it breaks a workflow operators are told to use.

HONEST BOUNDARY. This is not a live data defect. Zero corpus instances, and the detective gate catches it at the next validate. It is filed so a scoped, deliberate omission stays visible as a tracked decision instead of living only in a code comment where the next reader will mistake it for an oversight (§11.4.197: a started thread reaches a documented terminal state, never quiet abandonment).

PROVENANCE. Raised by the implementing agent of BOB-166 as a known asymmetry it deliberately did not close, independently verified and endorsed as an acceptable scope boundary by that item's reviewer, on the condition that it become a tracked item rather than a comment. This is that item.